

第8章 索引结构



第8章 索引结构



概述



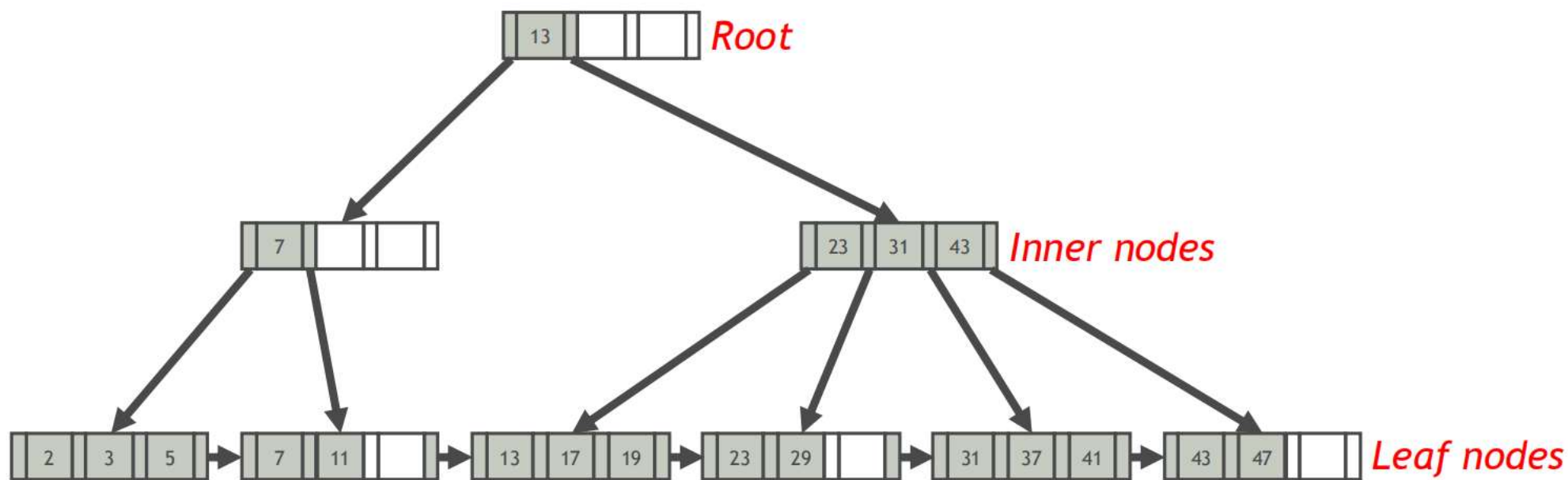
B+ 树索引结构



哈希索引结构

■ B+ 树索引

- B+ 树是大多数 RDBMS 所使用的索引结构
- B+ 树是一棵平衡多叉树，所有叶节点的深度都相同
- 索引项全部存储在 B+ 树的叶节点中，并按索引键值排序存储



■ 哈希索引

- 哈希索引是基于哈希表实现的
- 哈希表中的索引项是（索引键值的哈希值，元组地址）
- 哈希索引只支持对所有索引属性的精确匹配

Example (哈希索引)

哈希索引	
$h(\text{Sname})$	地址
111	$addr_2$
222	$addr_4$
333	$addr_3$
444	$addr_1$

● $h('Abby') = 333$

● $h('Ed') = 111$

Student 关系				
Sno	Sname	Ssex	Sage	Sdept
CS-001	Elsa	F	19	CS
CS-002	Ed	M	19	CS
MA-001	Abby	F	18	Math
PH-001	Nick	M	20	Physics

● $h('Elsa') = 444$

● $h('Nick') = 222$

■ 哈希索引的限制

CREATE INDEX idx ON Student (Sname, Sage, Sex) **USING HASH**;

- 哈希索引不支持部分索引属性匹配

SELECT * FROM Student WHERE Sage = 19; (不能使用索引)

- 哈希索引只支持等值比较查询 (=, IN) , 不支持范围查询

SELECT * FROM Student WHERE Sname = 'Elsa' AND Ssex = 'F'
AND Sage < 19; (不能使用索引)

- 哈希索引并不是按照索引值排序存储的, 所以无法用于排序
- 哈希索引存在冲突问题



概述



B+ 树索引结构

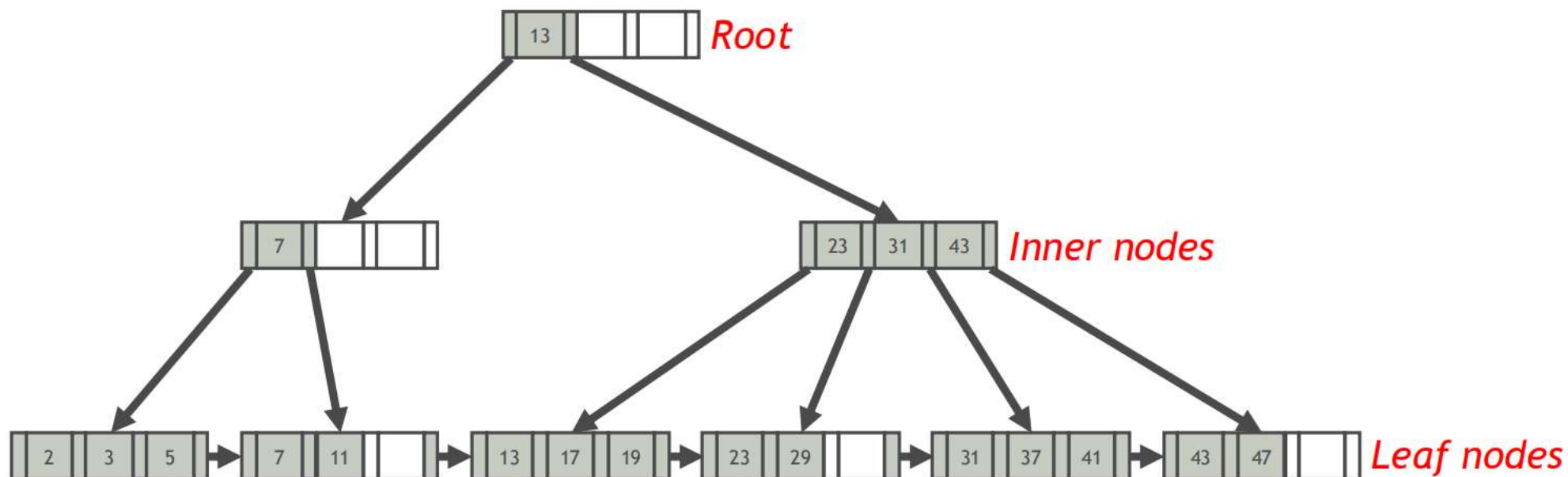


哈希索引结构

■ B+ 树

B+ 树是一种 M 路搜索树，具有以下特点：

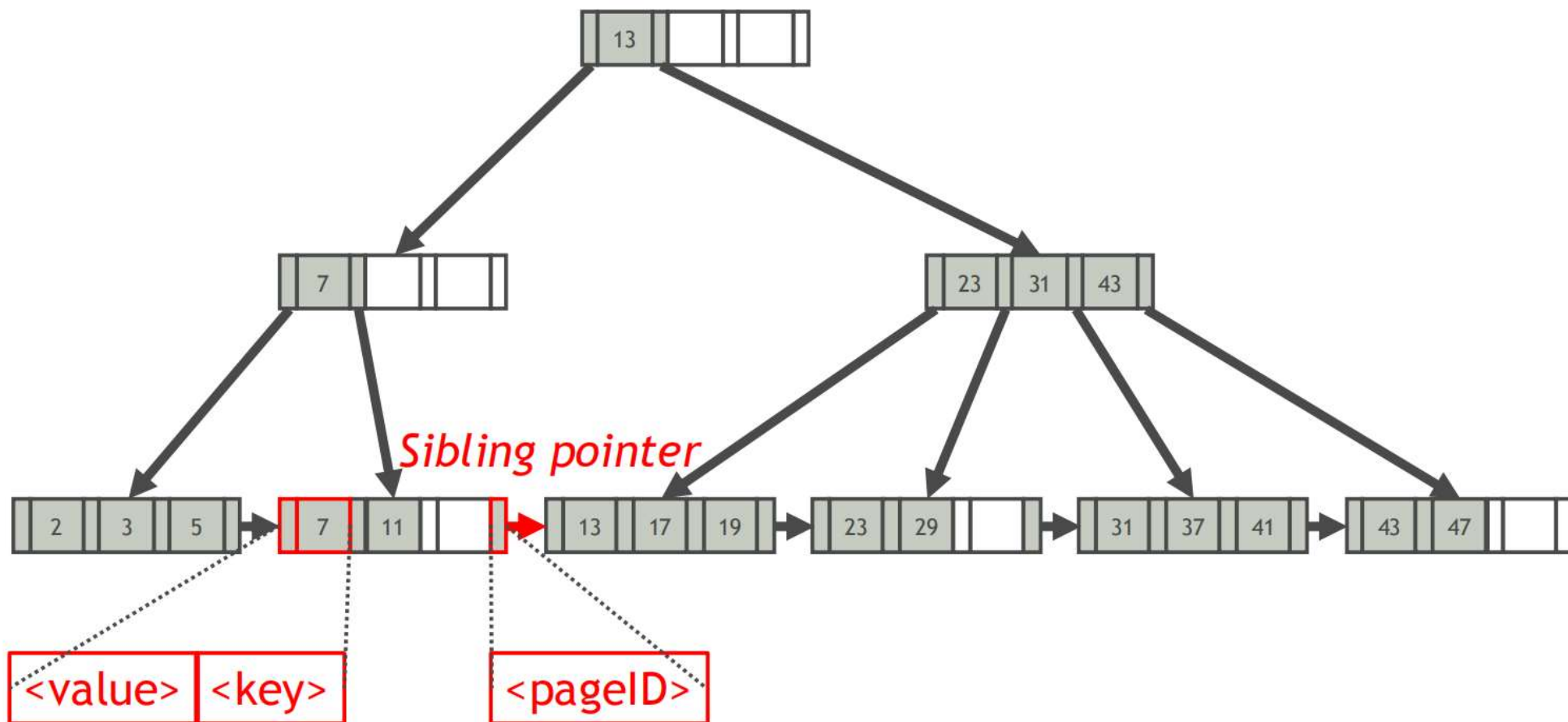
- 它是完美平衡的（即，每个叶节点都在相同的深度）
- 除根节点外的每个节点都至少半满 $M/2 - 1 \leq \#keys \leq M - 1$
- 每个具有 k 个键的内部节点具有 k + 1 个非空孩子
- 每个节点都适合一页



■ B+ 树叶子节点

每个叶节点由索引条目（键/值对）数组和指向其右兄弟的指针组成

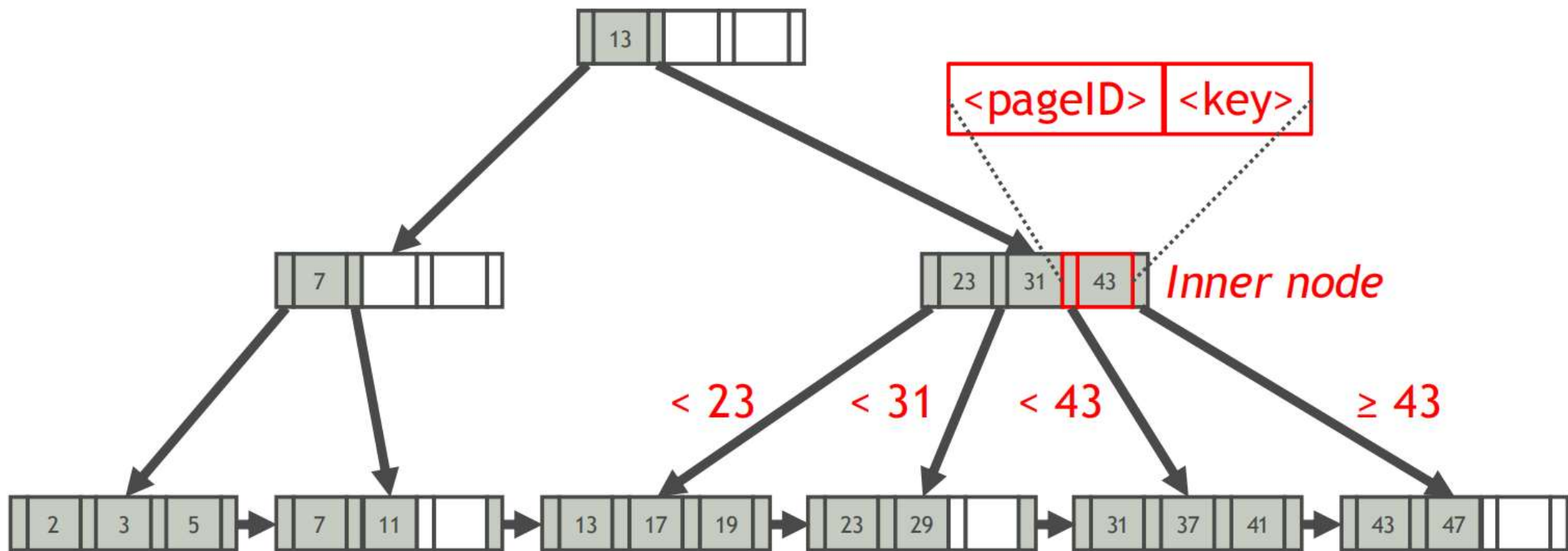
➤ 索引条目数组（通常）按索引键顺序存储



■ B+ 树内部节点

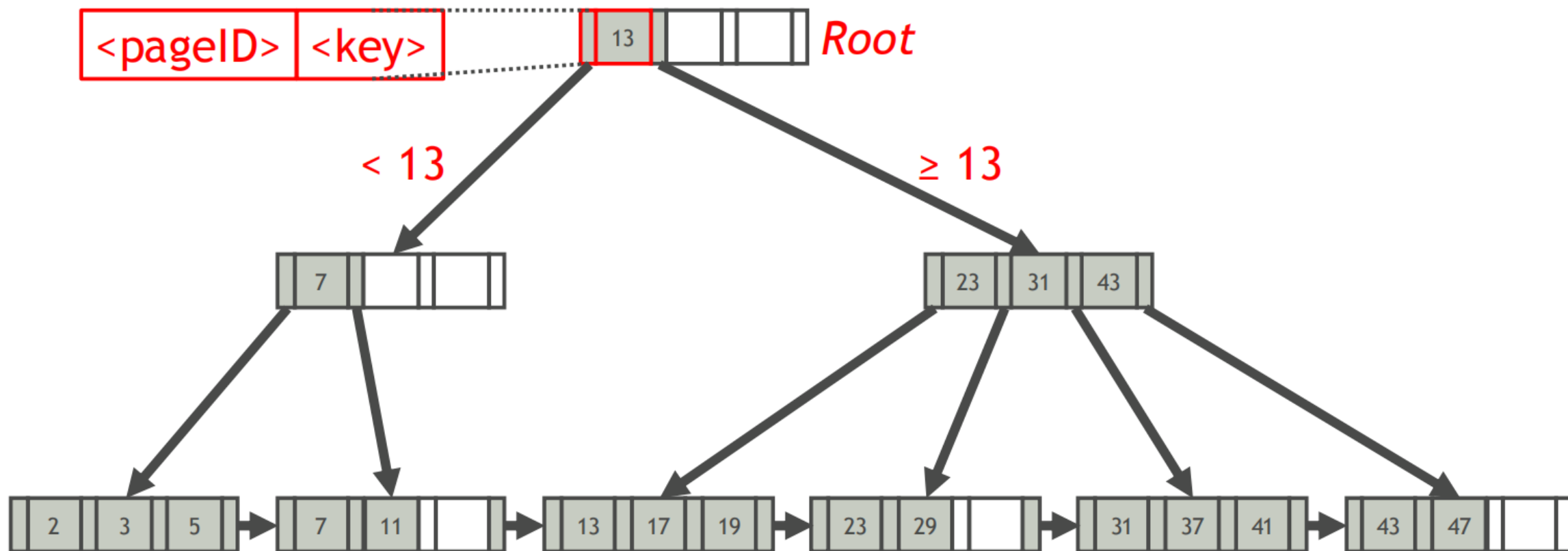
每个内部节点由键数组和指向其孩子的指针数组组成

- 键来自索引所基于的属性
- 数组（通常）按索引键顺序保存



■ B+ 树根节点

根节点包含至少一个键

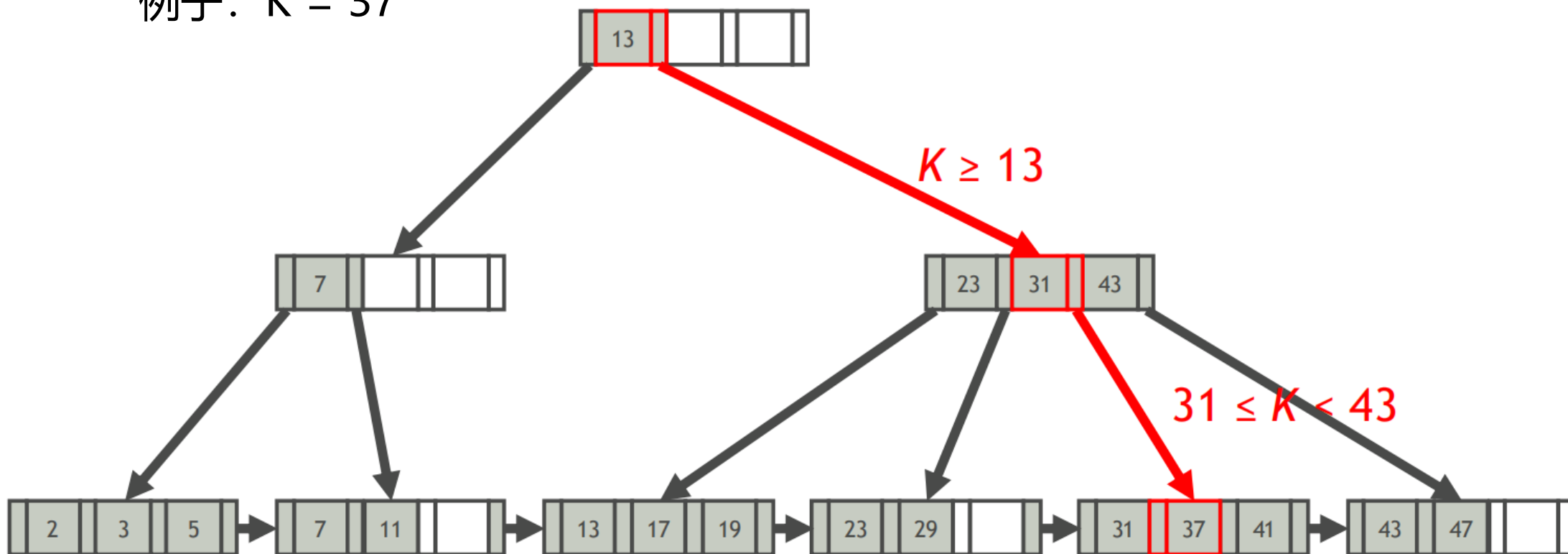


■ B+ 树查找

查找具有键 K 的索引条目

- 按内部节点中键的方向找到叶节点 K 所属的叶节点
- 在叶节点中查找具有键 K 的条目

例子: $K = 37$

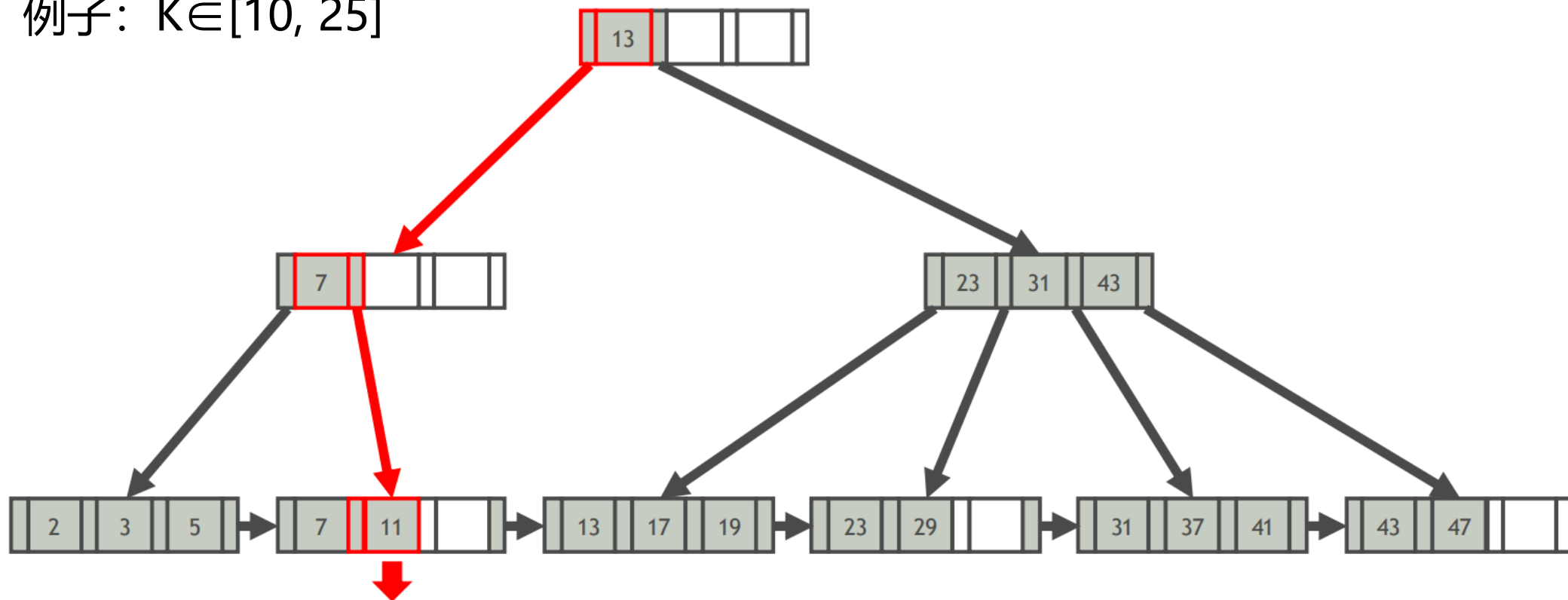


■ B+ 树范围查询

查找具有键 $K \in [L, U]$ 的索引条目

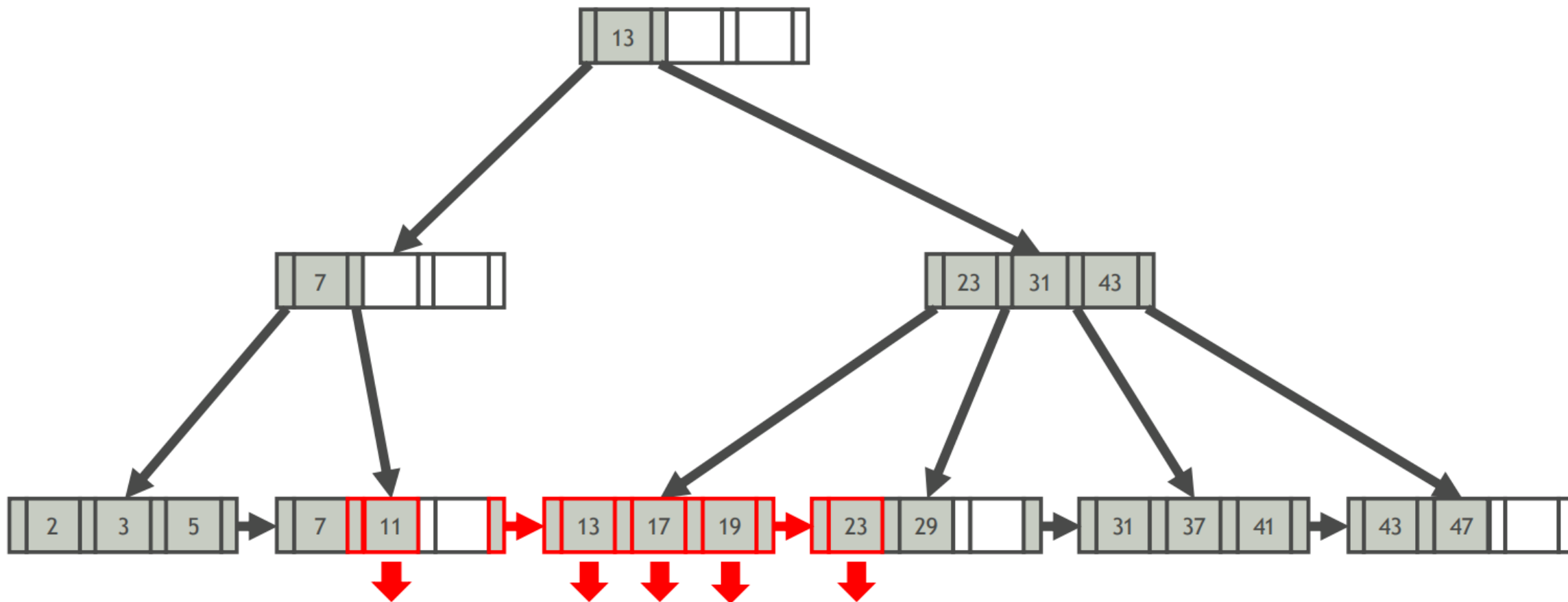
- 查找具有最小键 $\geq L$ 的第一索引条目 E
- 扫描 E 右侧键 $\leq U$ 的连续索引条目

例子: $K \in [10, 25]$



■ B+ 树范围查询 (续)

例子: $K \in [10, 25]$



■ B+ 树插入

插入具有键 K 的索引条目

- 找到要插入条目的正确叶节点 L
- 按索引键顺序将条目放入 L 中
- 如果 L 有足够的空间，完成！

否则，将 L 中的索引键分成 L 和一个新节点 L_2

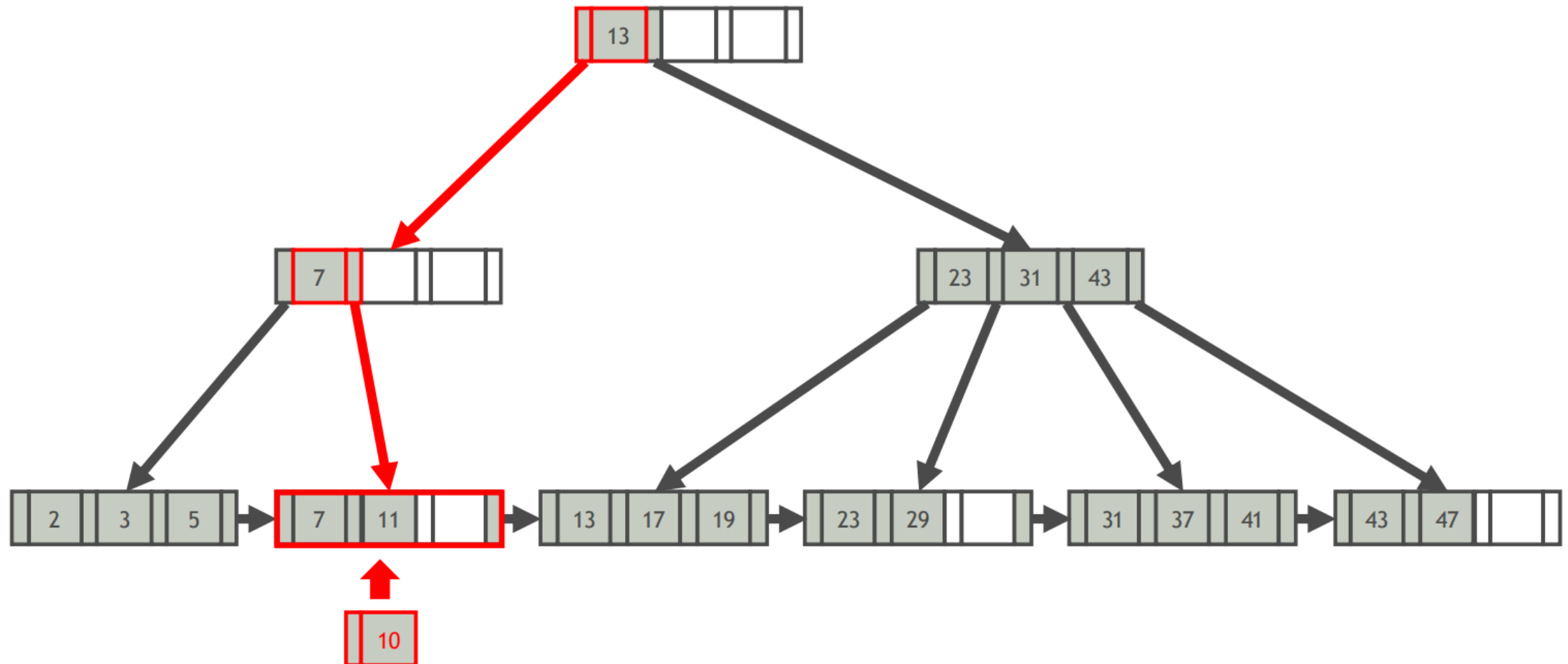
- 均匀分配条目，复制中间键
- 将指向 L_2 的索引条目插入 L 的父级中

如果要拆分内部节点

- 均匀重新分配条目
- 上推中间索引键

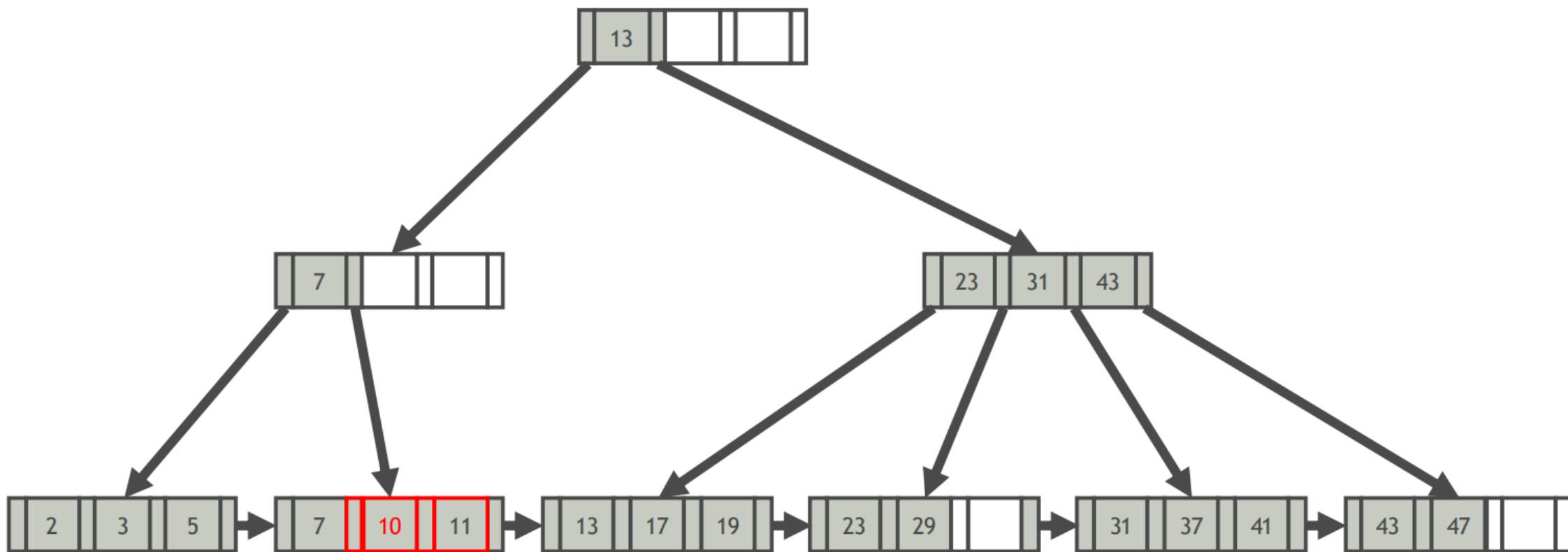
■ B+ 树插入：示例 1 (w/o 节点分裂)

例子：k = 10



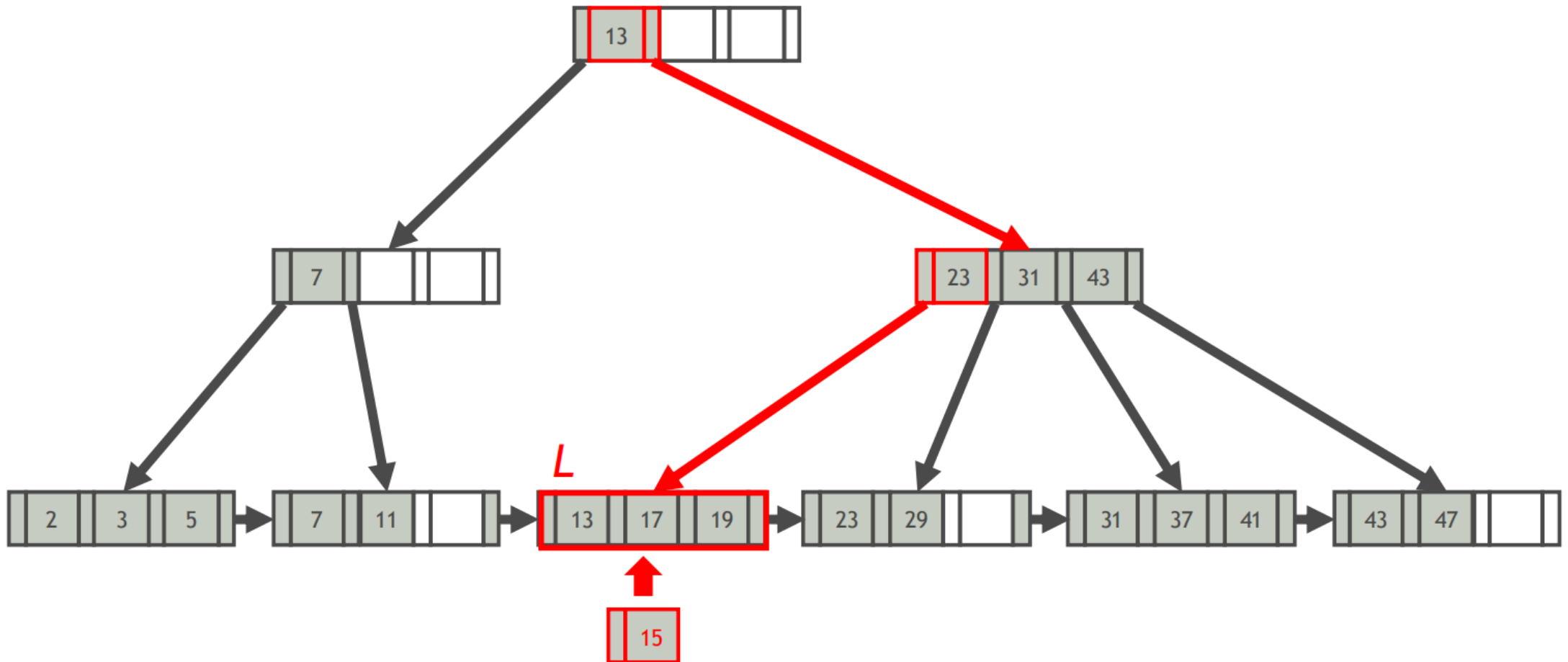
■ B+ 树插入：示例 1 (w/o 节点分裂)

例子: $k = 10$



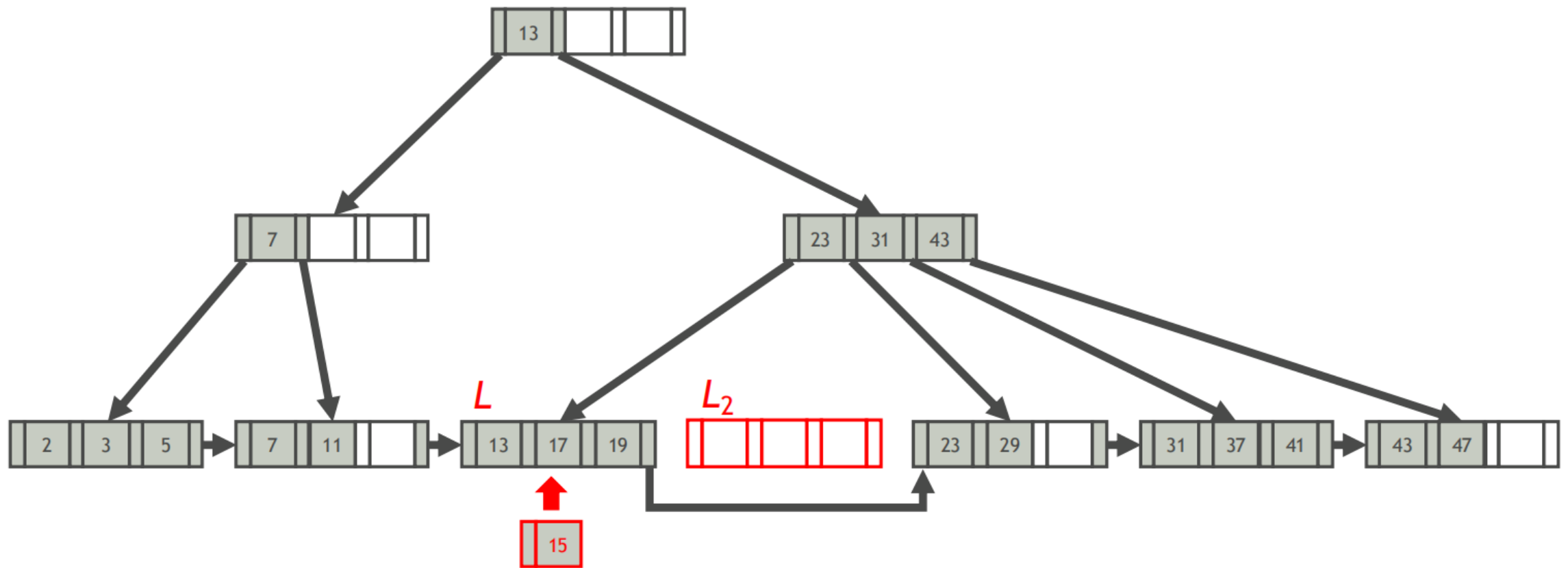
■ B+ 树插入：示例 2 (w/ 节点分裂)

例子: $k = 15$



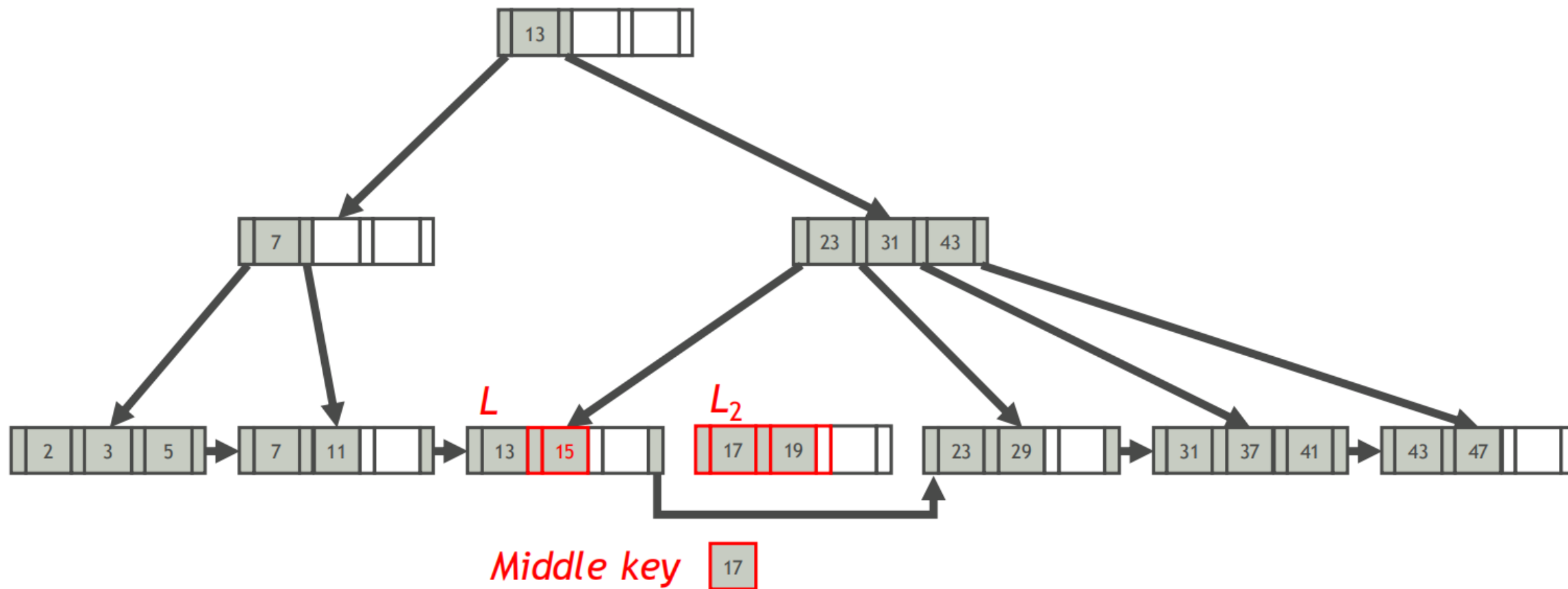
■ B+ 树插入：示例 2 (w/ 节点分裂)

例子: $k = 15$



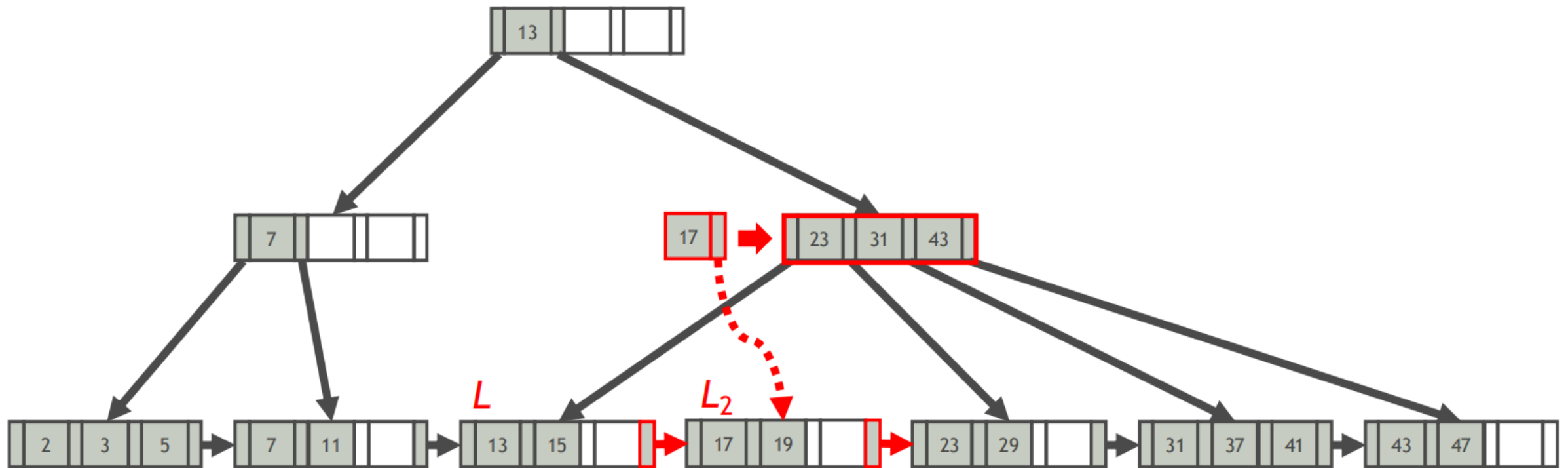
■ B+ 树插入：示例 2 (w/ 节点分裂)

例子: $k = 15$



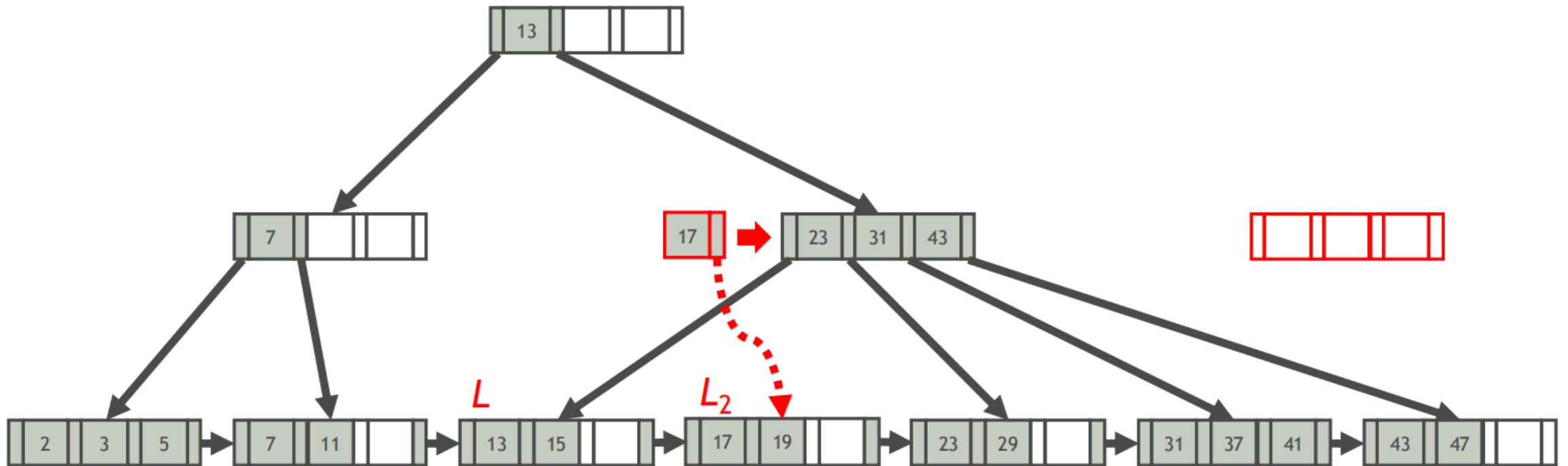
■ B+ 树插入：示例 2 (w/ 节点分裂)

例子: $k = 15$



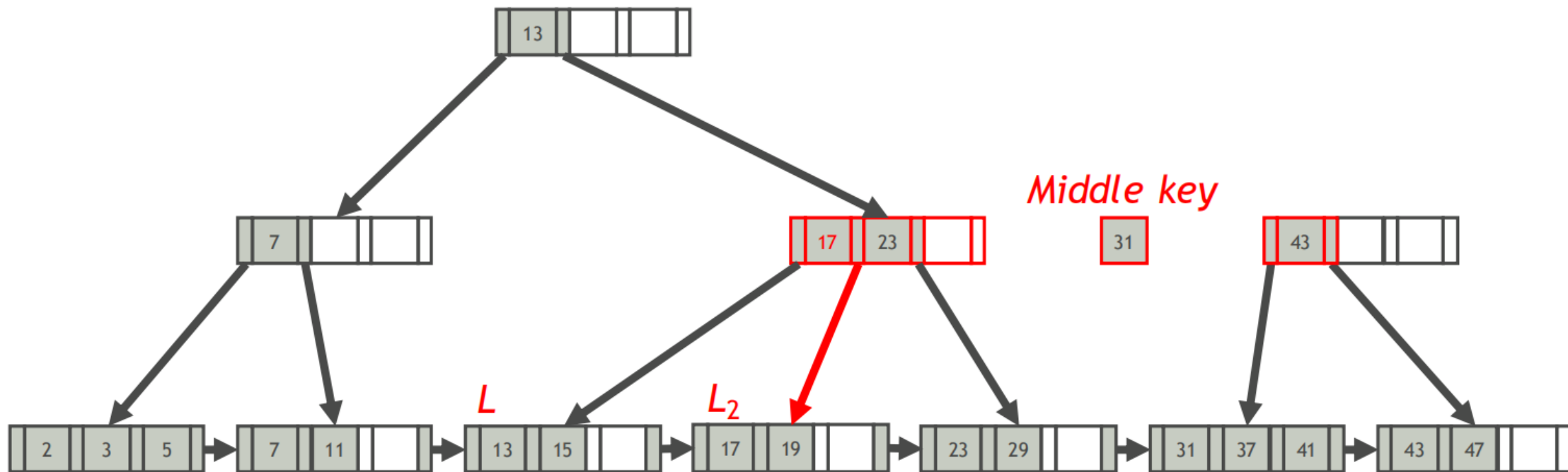
■ B+ 树插入：示例 2 (w/ 节点分裂)

例子: $k = 15$



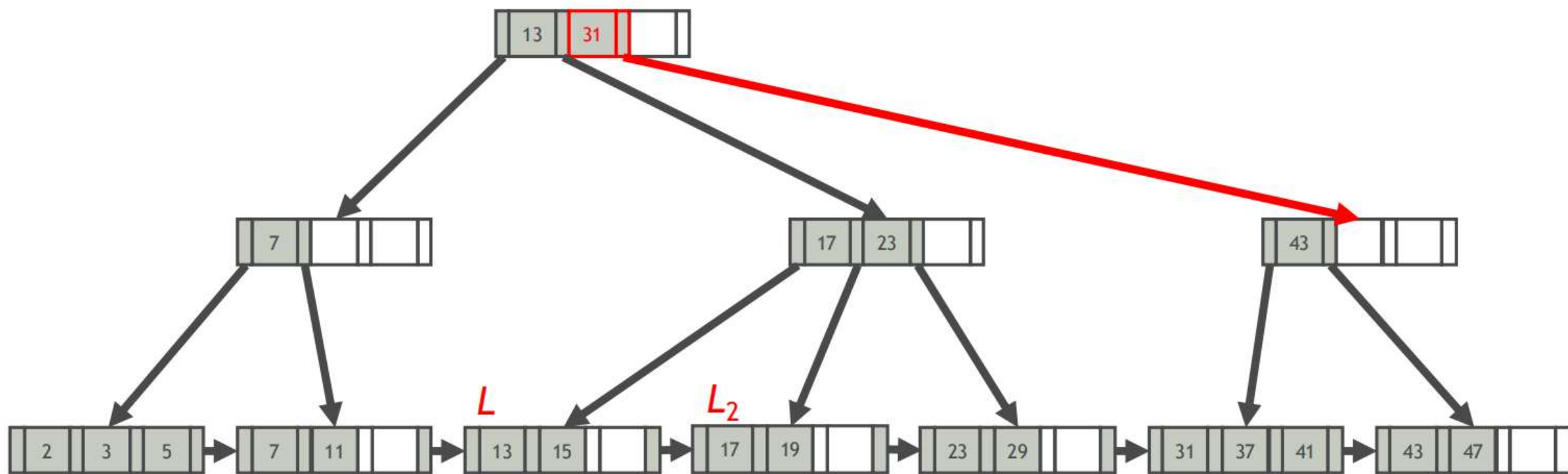
■ B+ 树插入：示例 2 (w/ 节点分裂)

例子: $k = 15$



■ B+ 树插入：示例 2 (w/ 节点分裂)

例子: $k = 15$



■ B+ 树删除

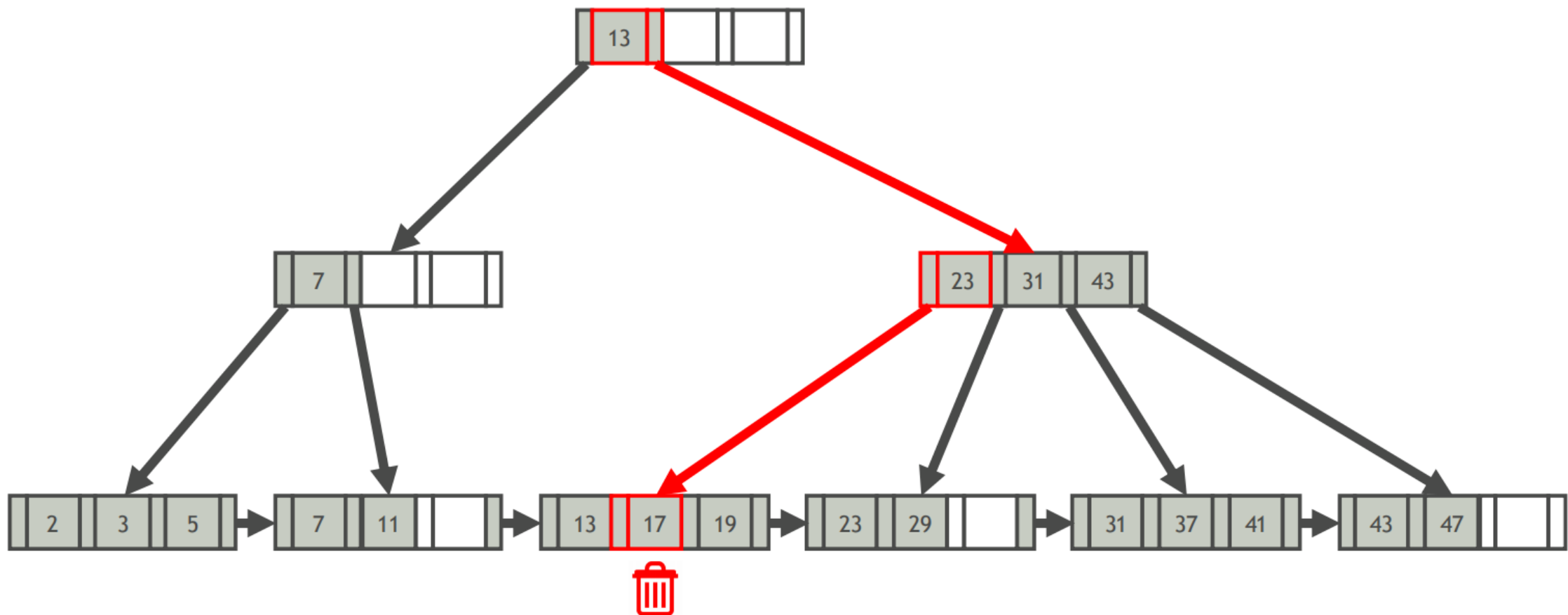
删除一个键值为 K 的索引条目

- 找到该条目所属的叶节点 L
 - 从 L 中删除该条目
 - 如果 L 满足至少半满，则完成
- 否则，
- 尝试重分布，从兄弟节点中借用
 - 如果重分布失败，则合并 L 和其兄弟节点

如果发生合并，则必须从 L 的父节点中删除指向 L 或其兄弟节点的条目

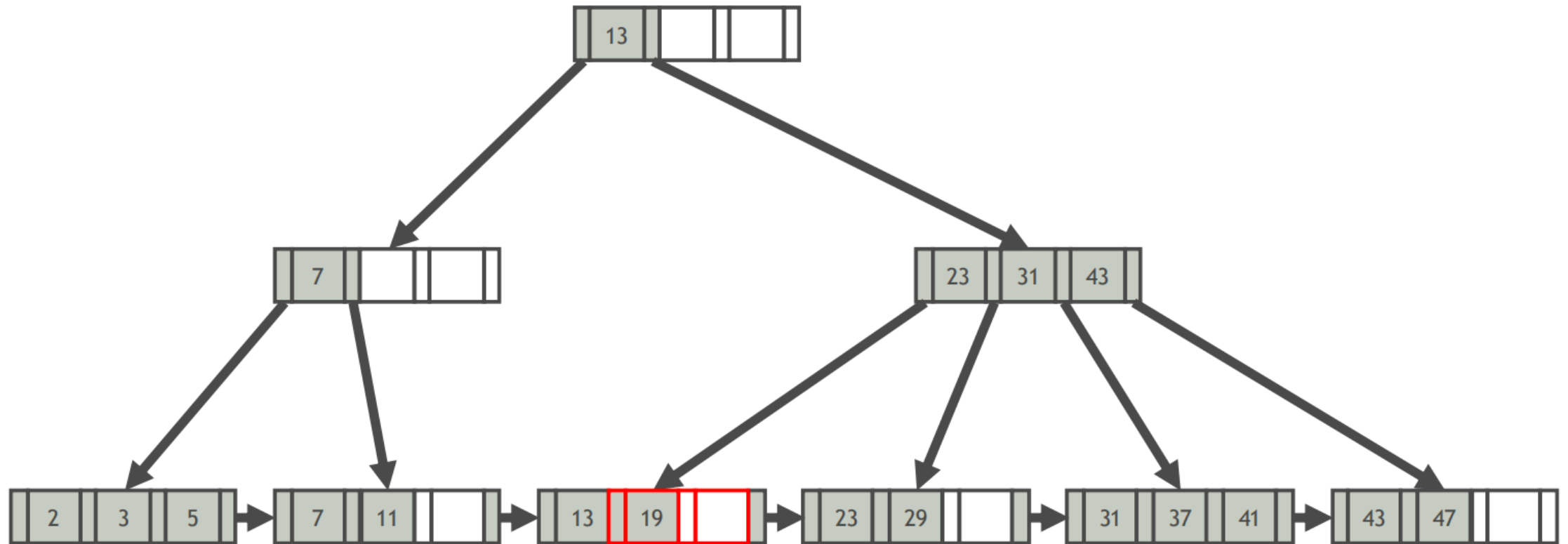
■ B+ 树删除：示例 1 (w/o 节点合并)

例子：k = 17



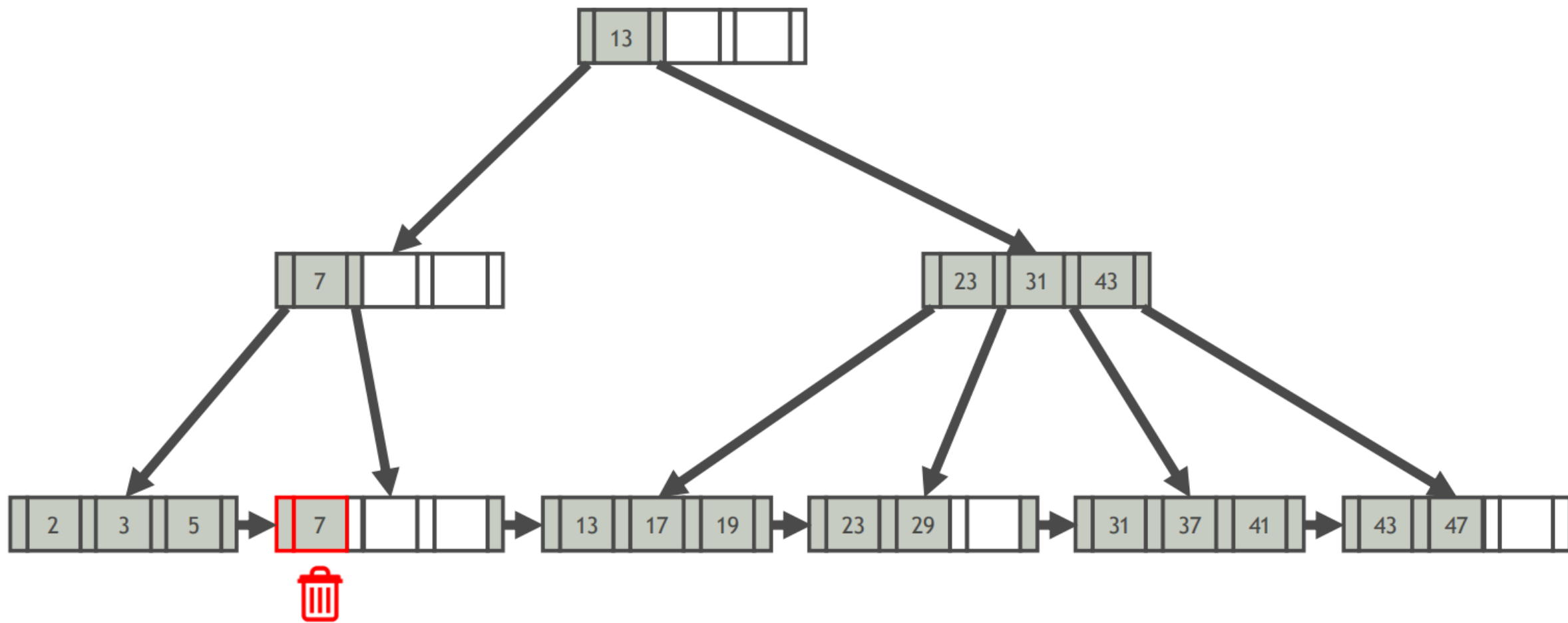
■ B+ 树删除：示例 1 (w/o 节点合并)

例子：k = 17



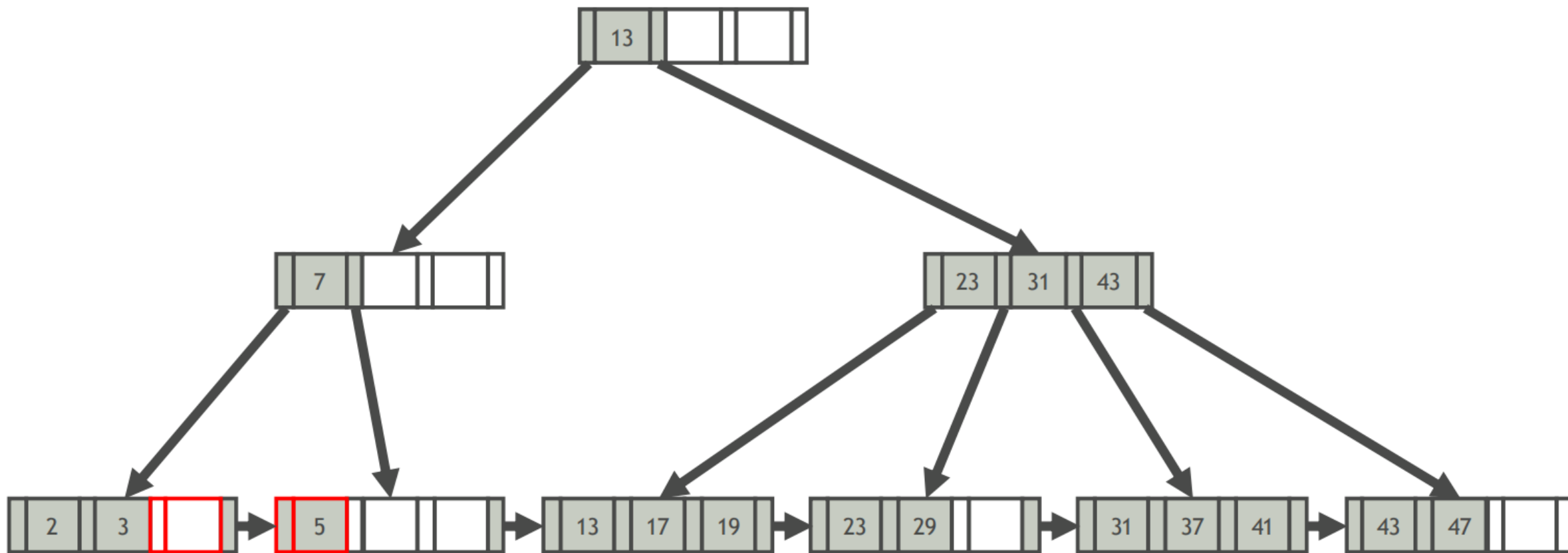
■ B+ 树删除：示例 2（键重新分配）

例子：k = 7



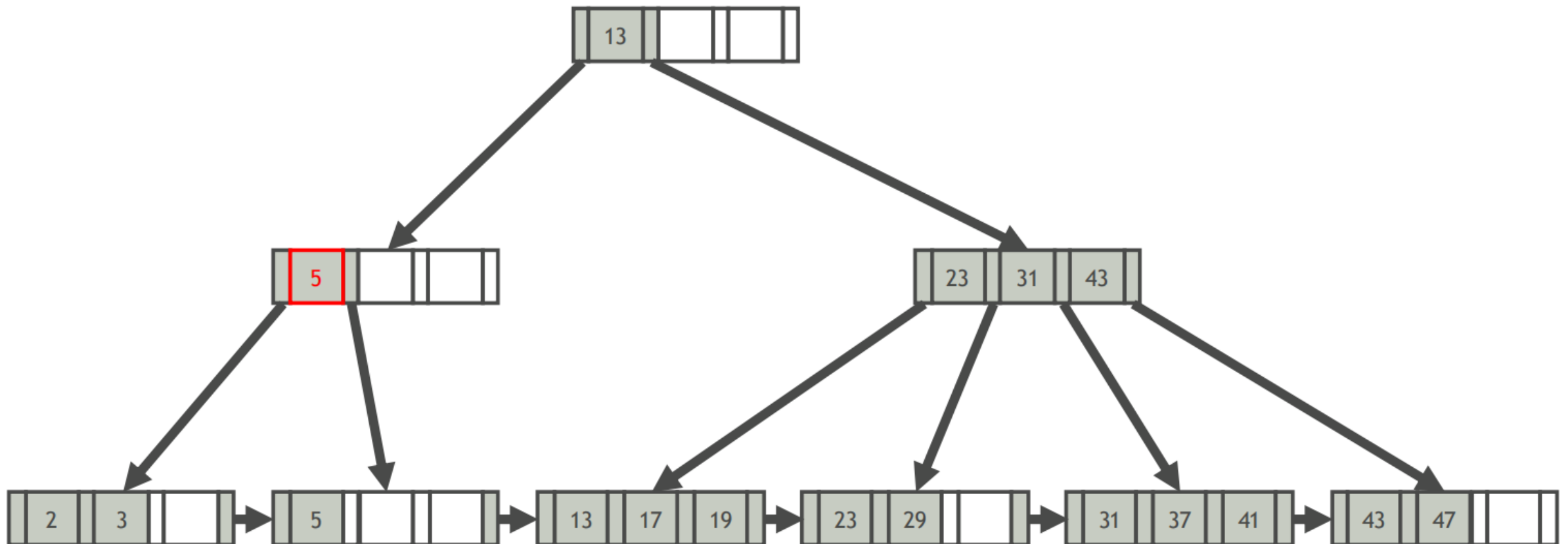
■ B+ 树删除：示例 2（键重新分配）

例子：k = 7



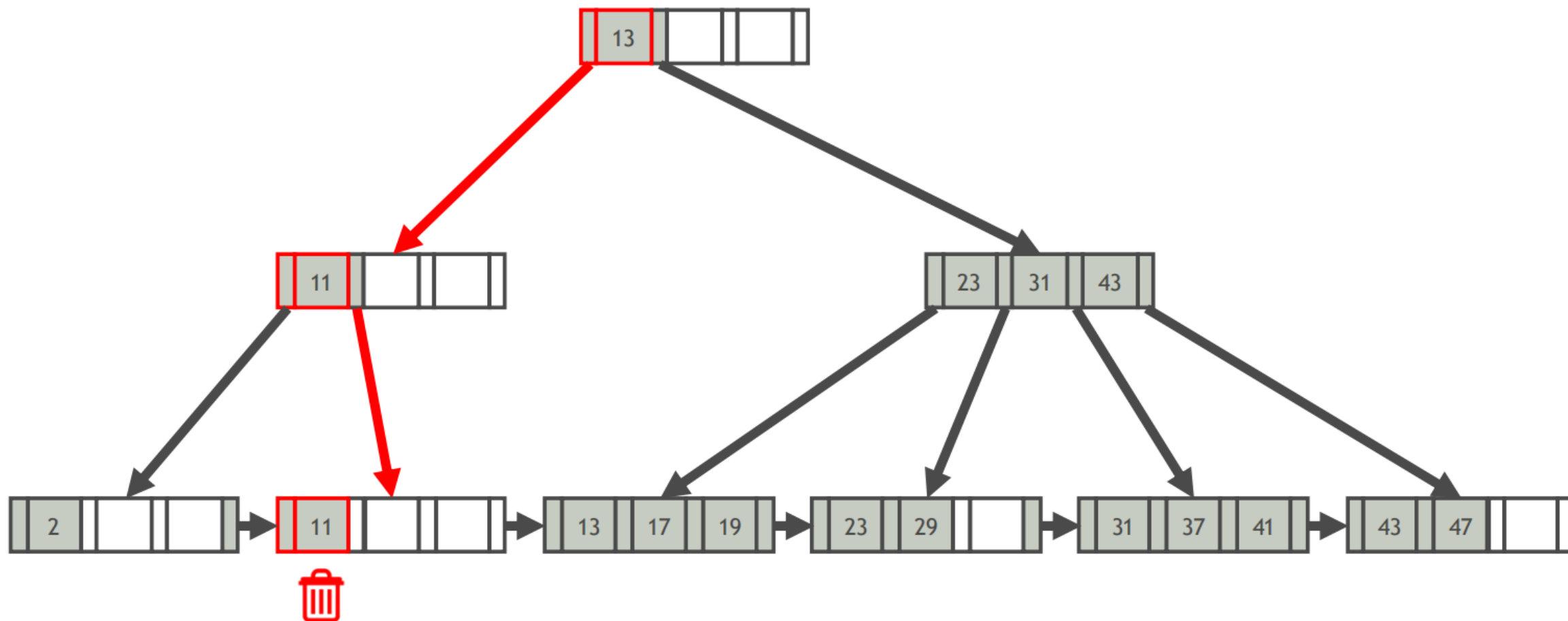
■ B+ 树删除：示例 2（键重新分配）

例子：k = 7



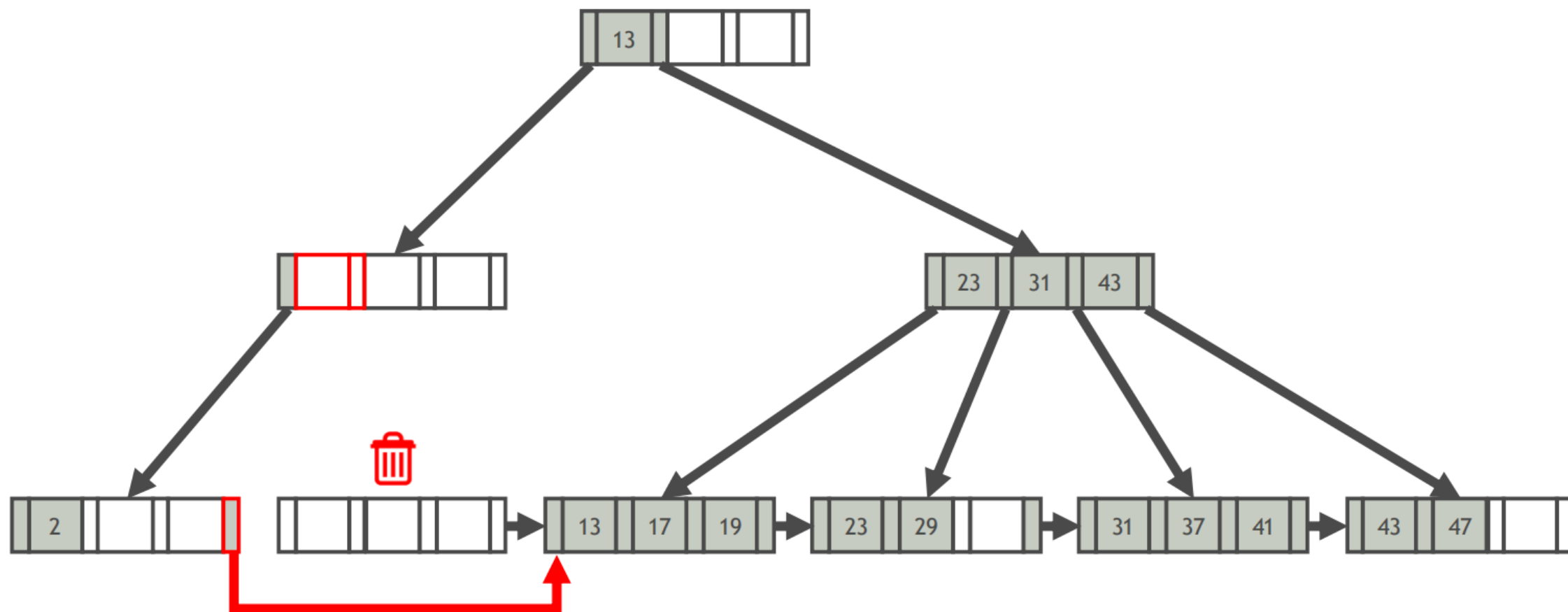
■ B+ 树删除：示例 3 (w/ 结点合并)

例子: $k = 11$



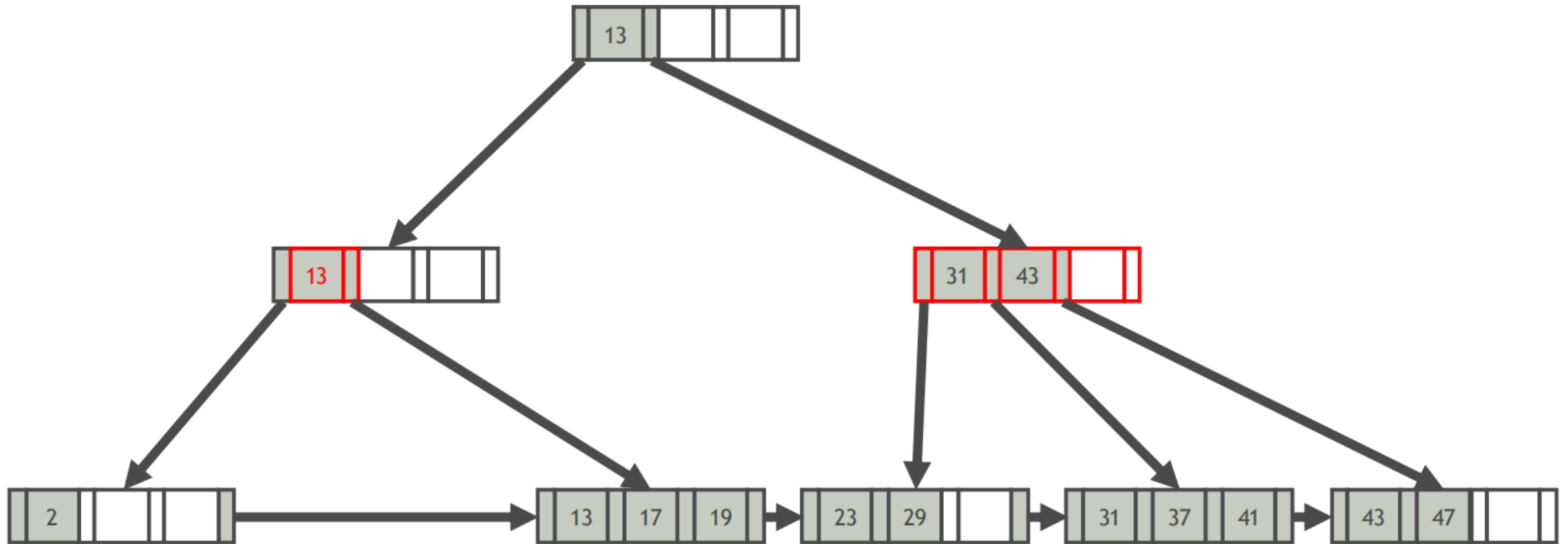
■ B+ 树删除：示例 3 (w/ 结点合并)

例子: $k = 11$



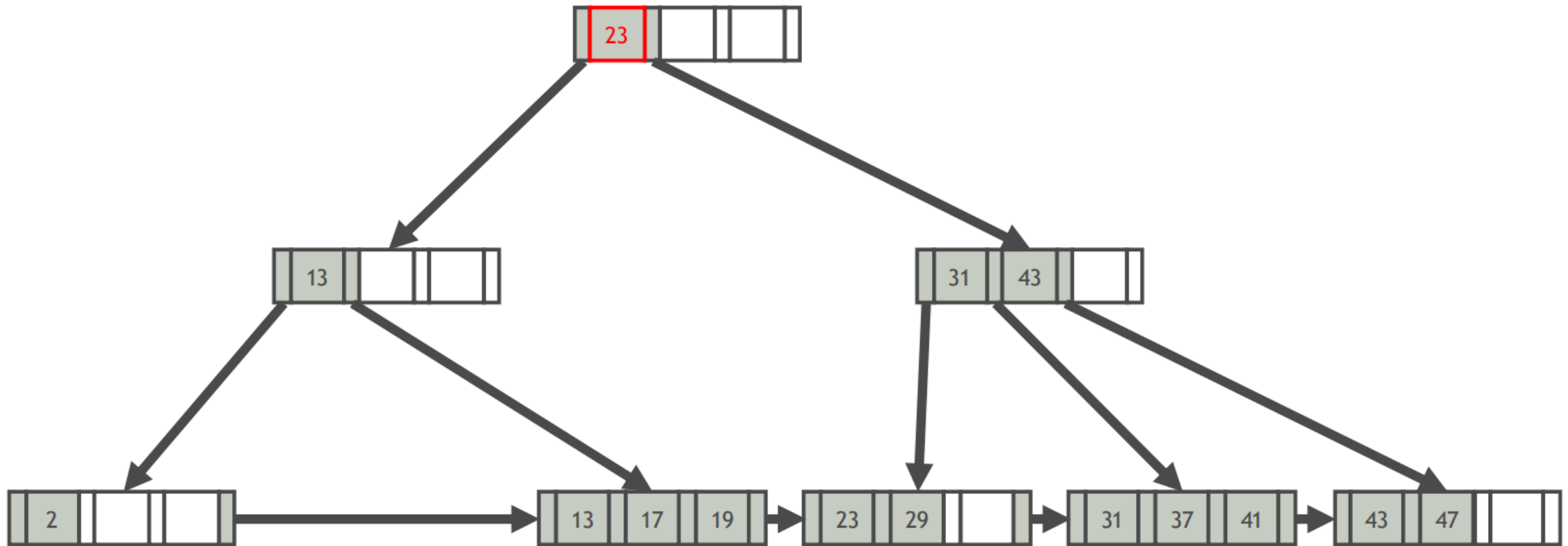
■ B+ 树删除：示例 3 (w/ 结点合并)

例子: $k = 11$



■ B+ 树删除：示例 3 (w/ 结点合并)

例子: $k = 11$



■ 键压缩

在 B+ 树中检索数据条目所需的磁盘 I/O 次数 = 树的高度 $\approx \log_{\text{fan_out}}(\text{条目数})$

- 树的扇出是指适合在一页上的索引条目的数量，这取决于索引条目的大小
- 索引条目的大小主要取决于搜索键值的大小

搜索键值非常长 $\Rightarrow$ 扇出较小 $\Rightarrow$ 树较高 $\Rightarrow$ 查询时间较长

■ 前缀压缩

- 在相同叶节点中排序的键通常具有相同的前缀
- 因此不必每次都存整个键，而是提取公共前缀，并仅为每个键存储唯一后缀

Microphone	Microsoft	Microwave
------------	-----------	-----------

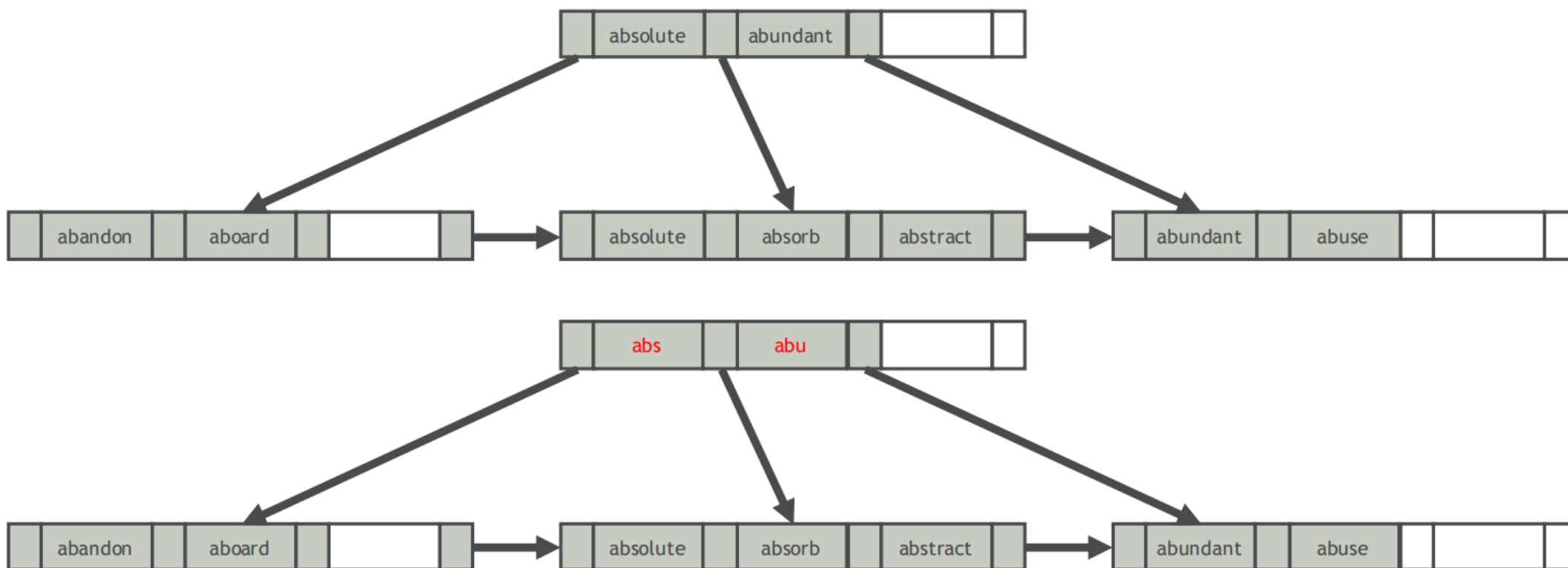
↓ Prefix compression

Prefix: Micro

phone	soft	wave
-------	------	------

■ 后缀截断

- 内部节点中的键仅用于路由
- 因此不必在内部节点中存储完整的键
- 而只需存储正确路由查询所需的最小前缀



■ 批量加载

在现有的索引条目集合中创建 B+ 树

自顶向下的方法

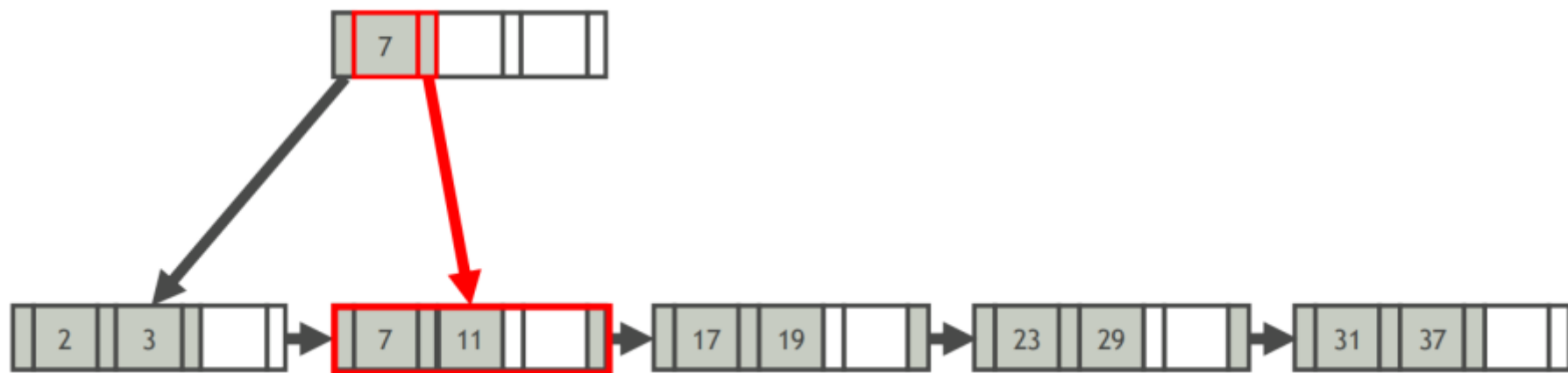
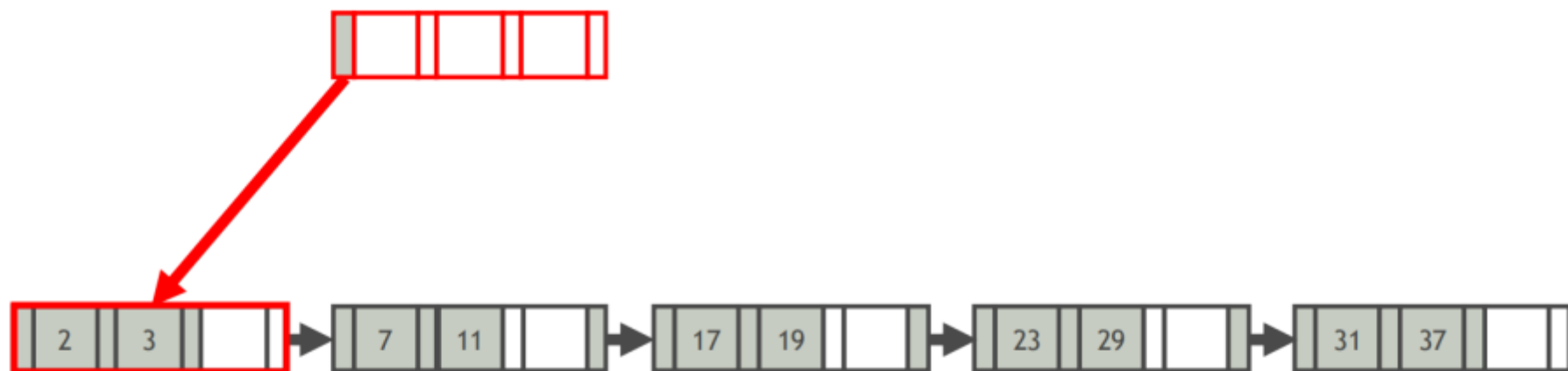
- 逐个插入索引条目
- 昂贵，因为每个条目都需要从根开始并下到适当的叶节点

自底而上的方法

- 按搜索键排序索引条目
- 分配一个空的内部节点作为根，并将指向已排序条目第一页的指针插入其中
- 叶页的条目始终插入最靠右的内部节点，正好在叶级别上方。当该页填满时，它会被分割

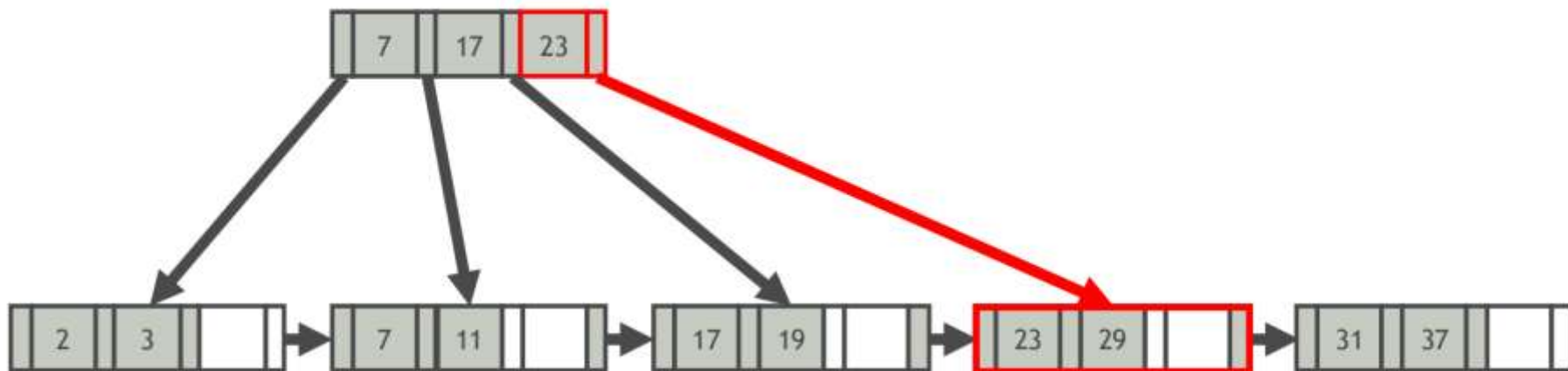
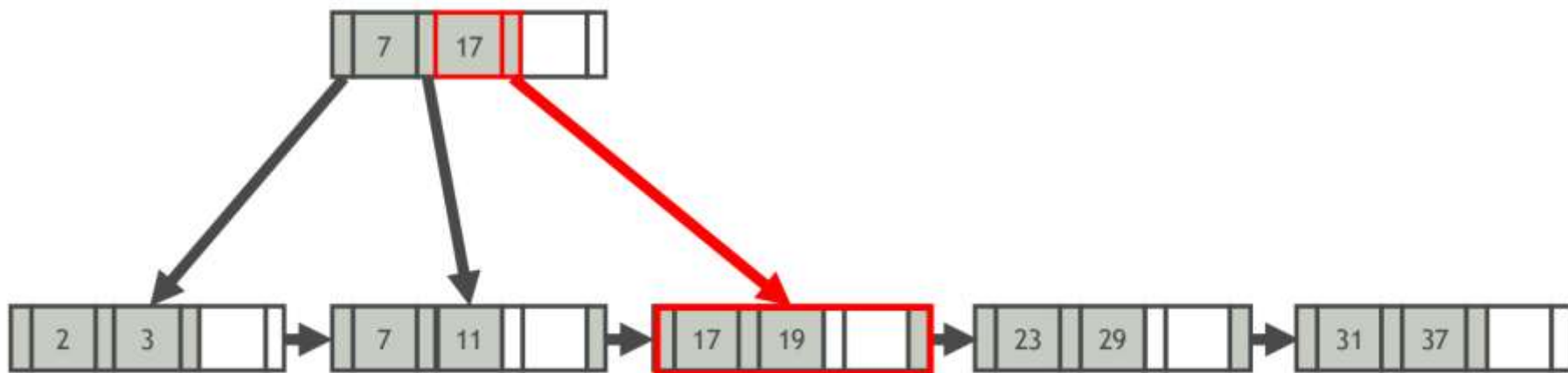
■ 批量加载：示例

排序键：2、3、7、11、17、19、23、29、31、37



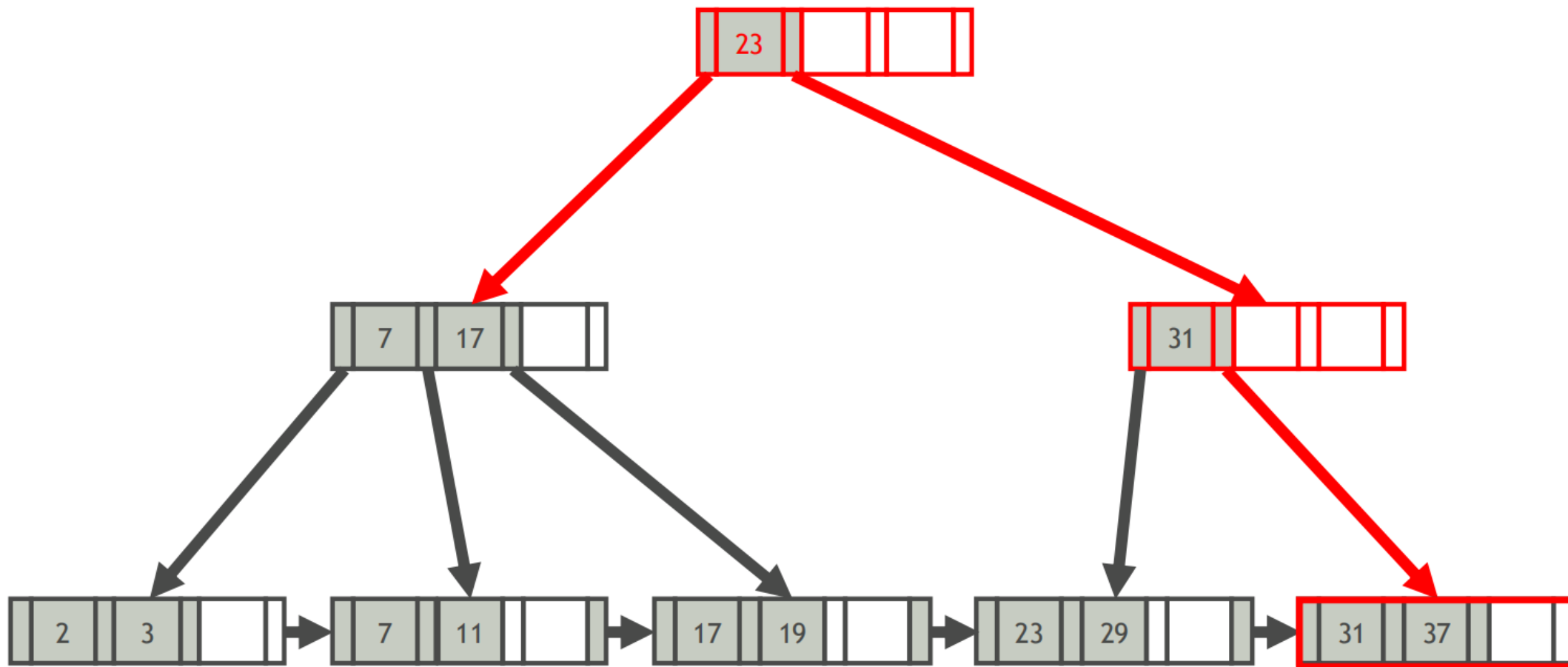
■ 批量加载：示例

排序键：2、3、7、11、17、19、23、29、31、37



■ 批量加载：示例

排序键：2、3、7、11、17、19、23、29、31、37





概述



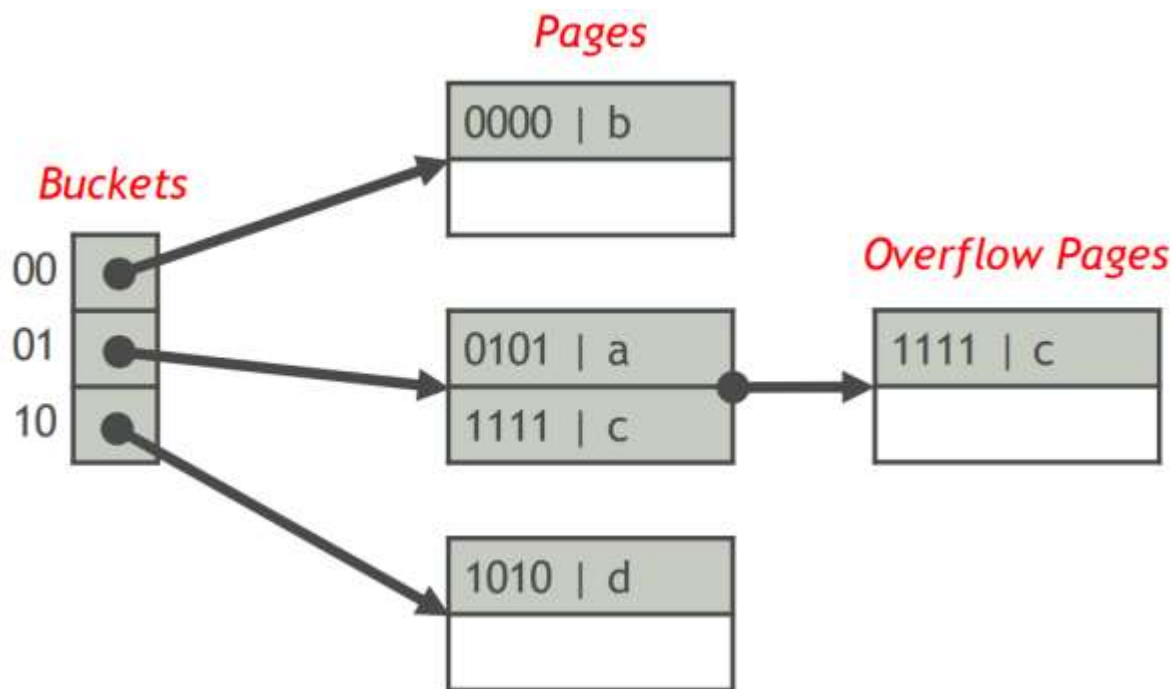
B+ 树索引结构



哈希索引结构

■ 外存哈希表

- 外存哈希表由若干个桶组成
- 带有关键字 K 的索引条目放入编号为 $\text{hash}(K)$ 的桶中，其中 hash 是一个哈希函数
- 每个桶存储指向存储在该桶中的索引条目的页面链表的指针



■ 外存哈希表的分类

- 静态哈希表

 - 桶的数量不变

- 动态哈希表

 - 桶的数量可以变化，以便每个桶大约有一个块

 - 可扩展哈希表

 - 线性哈希表

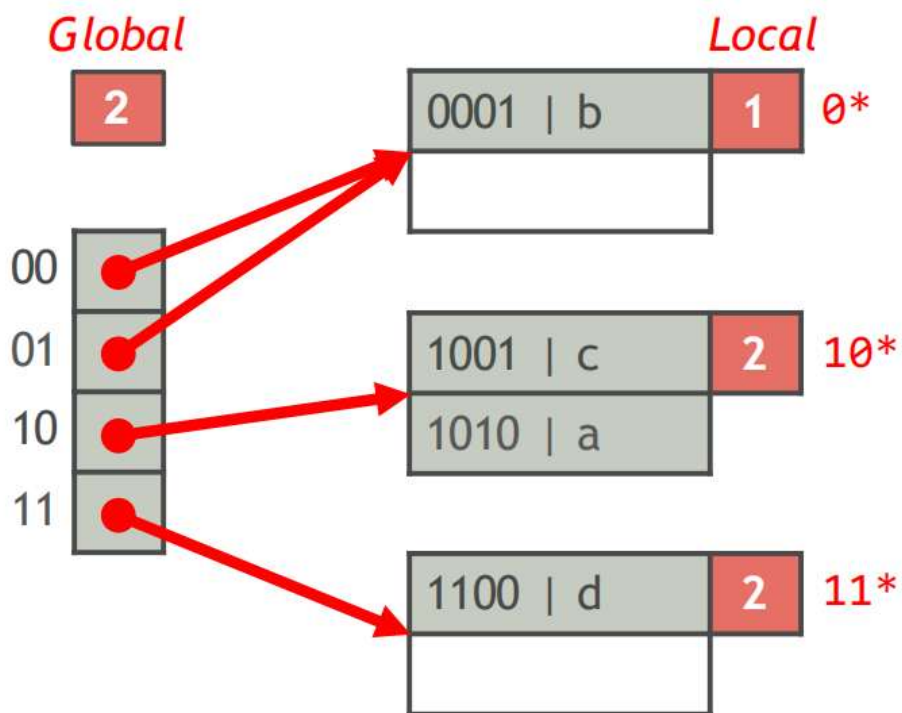
可扩展哈希表

■ 可扩展哈希表

可扩展哈希表由 2^i 个桶组成

- i 被称为全局深度
- 带有关键字 K 的索引条目属于 $\text{hash}(K)$ 的第一个 i 位指定的桶

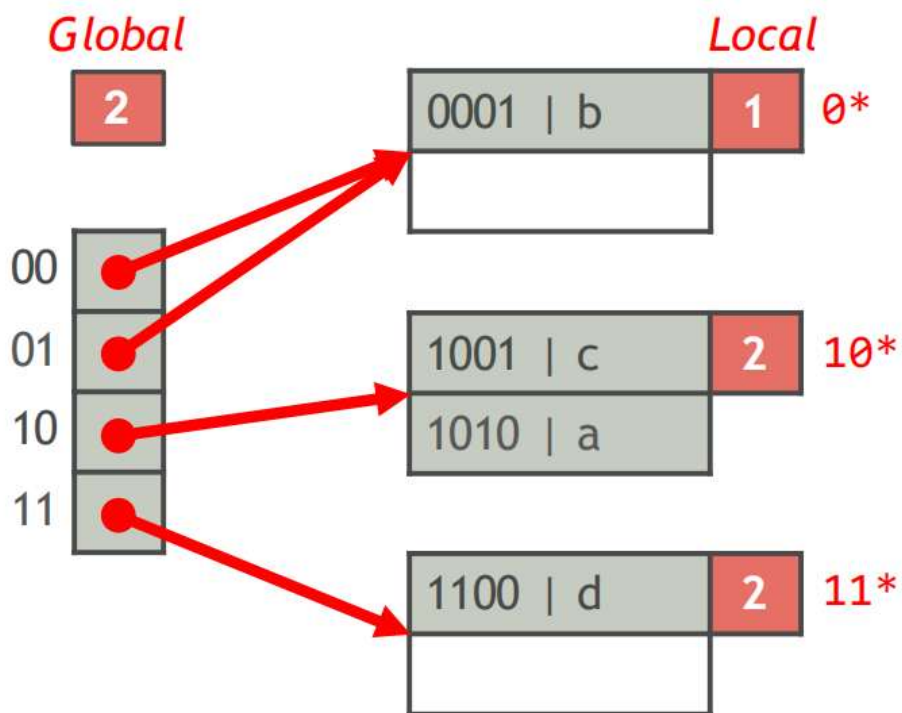
例如: $\text{hash}(a) = 1010$, $\text{hash}(b) = 0001$, $\text{hash}(c) = 1001$, $\text{hash}(d) = 1100$



■ 可扩展哈希表 (续)

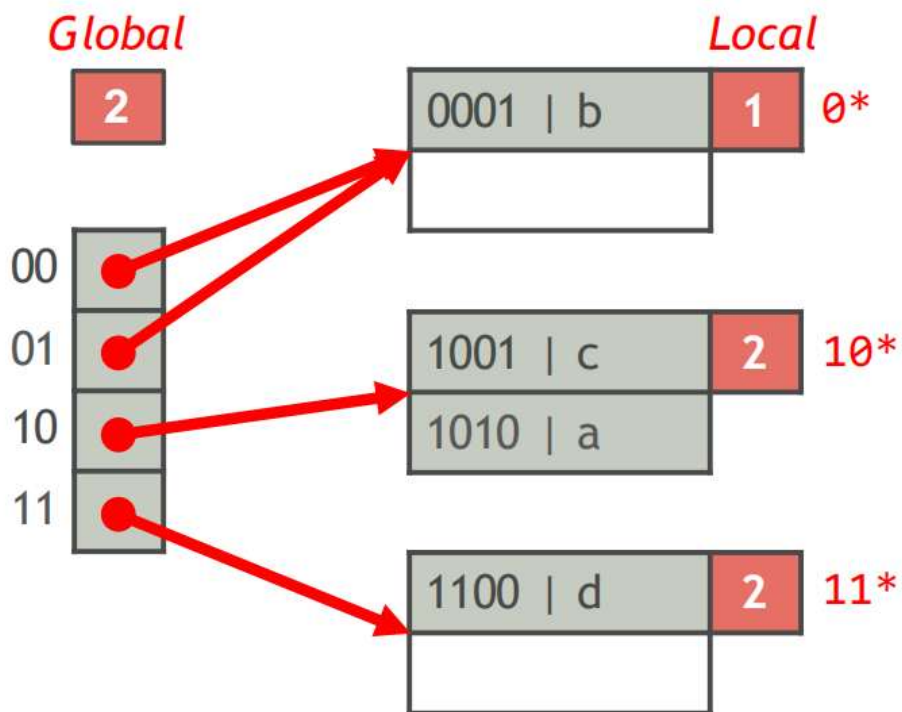
每个桶保留指向存储桶中索引条目的页面的指针

- 如果多个桶中的索引条目可以放入一页，则多个桶可以共享一页
- 每个页面记录 $\text{hash}(K)$ 的 #bits (本地深度) 用于确定索引条目成员身份



■ 可扩展哈希表 (续)

- $\#buckets = 2^{global_depth}$
- 全局深度必须大于或等于任何页面的本地深度
- 如果页面的本地深度小于全局深度，则仅在该页的本地深度小于全局深度时与另一个桶共享

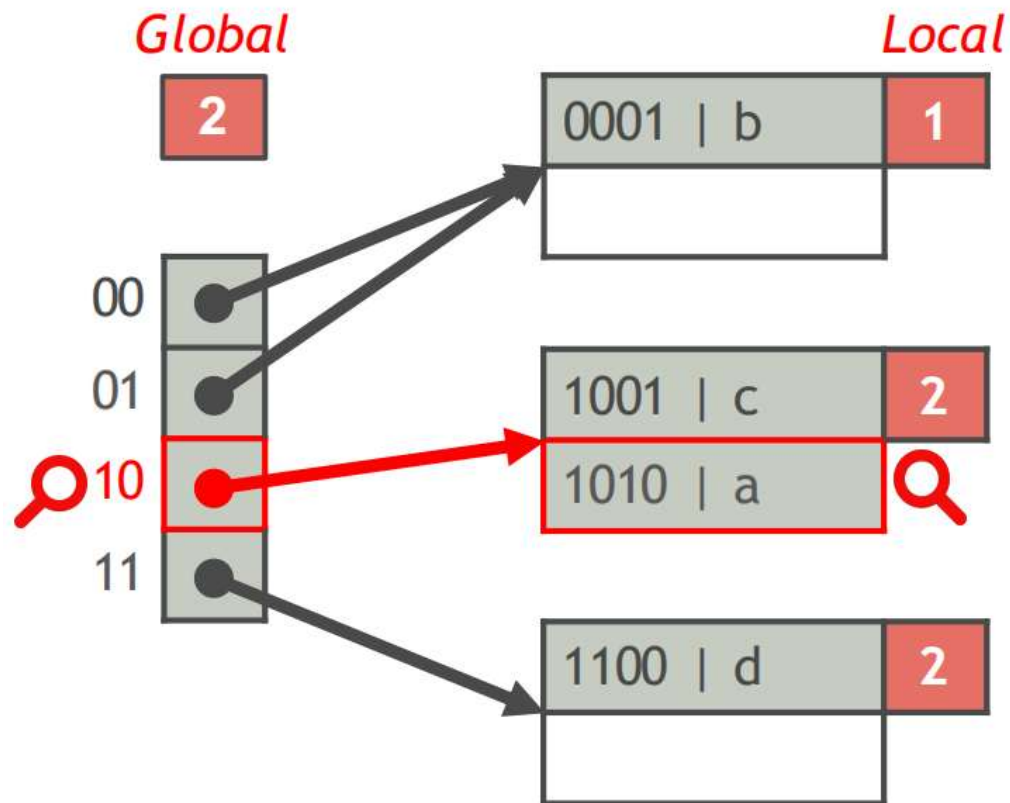


■ 可扩展哈希表查找

查找关键字 K 的索引条目

- 确定索引条目所属的桶
- 在桶指向的页面中查找条目

例如: $K = a$, $\text{hash}(a) = 1010$

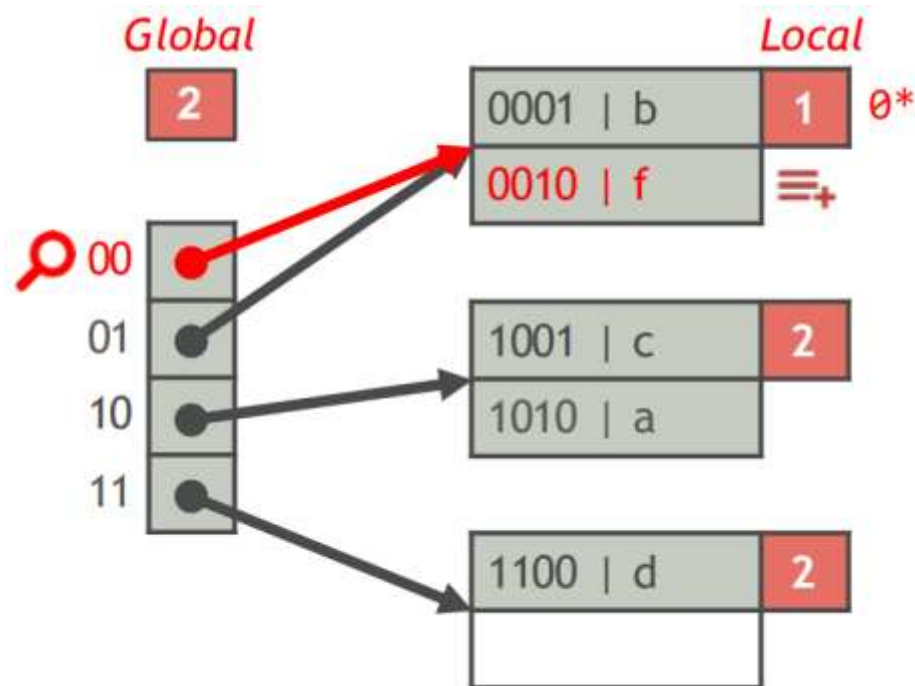


■ 可扩展哈希表插入

插入一个关键字为 K 的索引条目

- 找到要插入条目的页 P
- 如果 P 有足够的空间，完成。否则，将 P 分成 P 和一个新的页 P'

例如： $K = f$, $\text{hash}(f) = 0010$

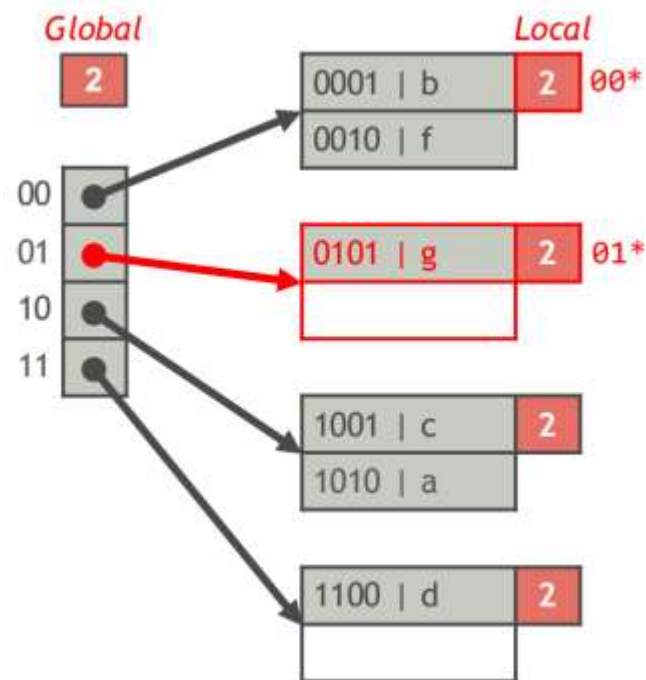
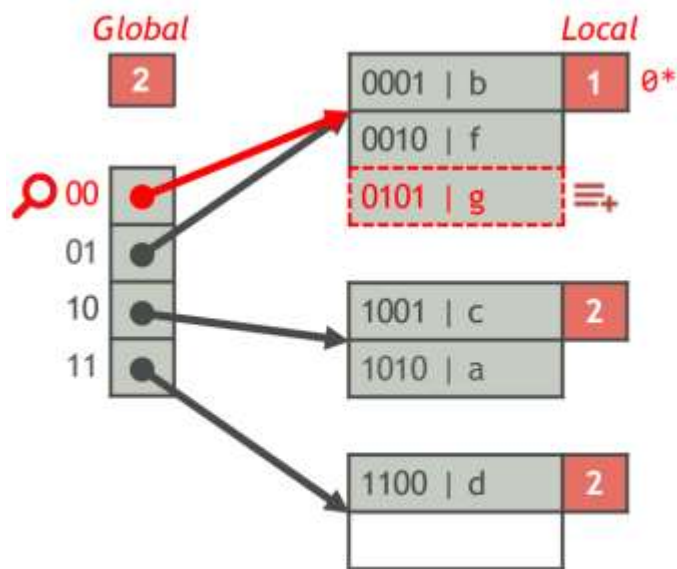


■ 可扩展哈希表插入 (续)

如果 P 溢出并且 P 的局部深度小于全局深度

- 将 P' 的局部深度增加 1
- 将 P 中的一些索引条目分配给新的桶页面 P' (P 和 P' 具有相同的局部深度)

例如: $K = g$, $\text{hash}(g) = 0101$

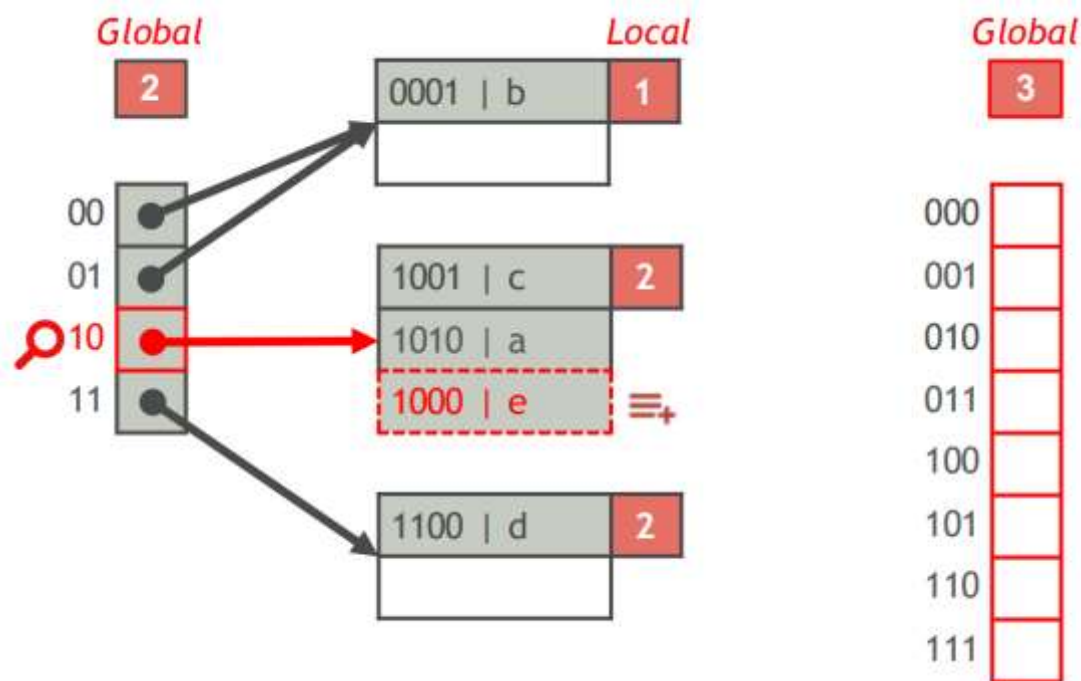


■ 可扩展哈希表插入 (续)

如果 P 溢出且 P 的本地深度等于全局深度,

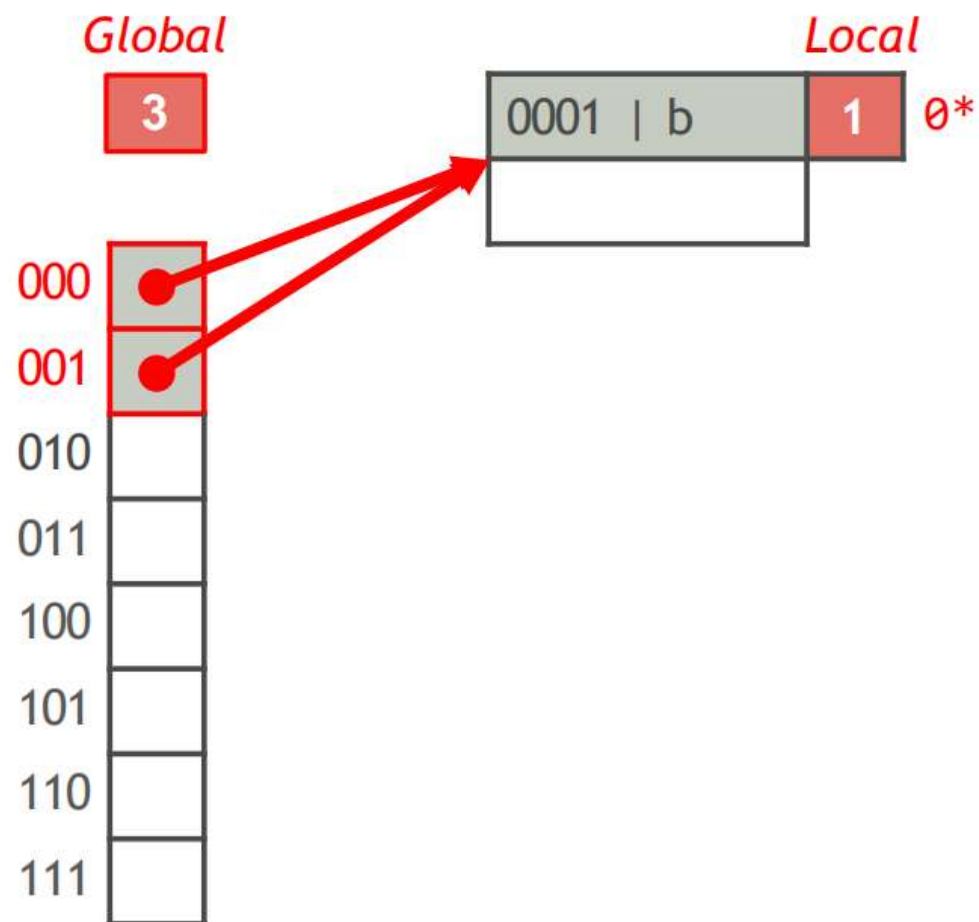
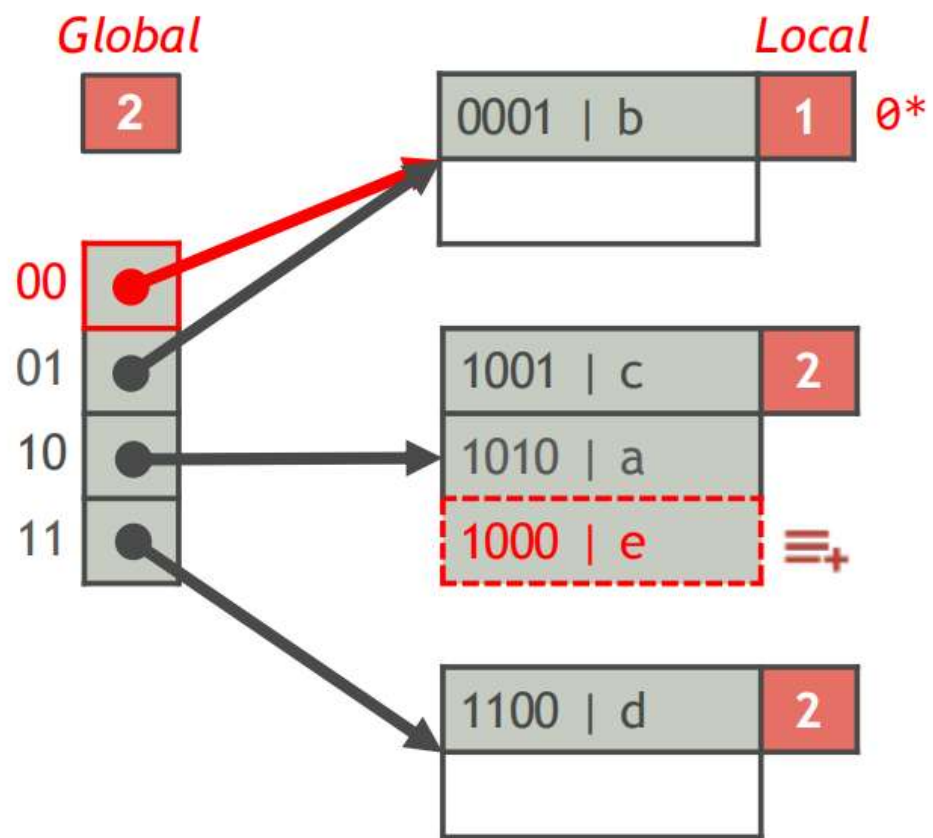
- 将全局深度增加 1 (桶的数量翻倍)
- 重新组织桶; 如果一个页面溢出, 则将其分割

例如: $K = e$, $\text{hash}(e) = 1000$



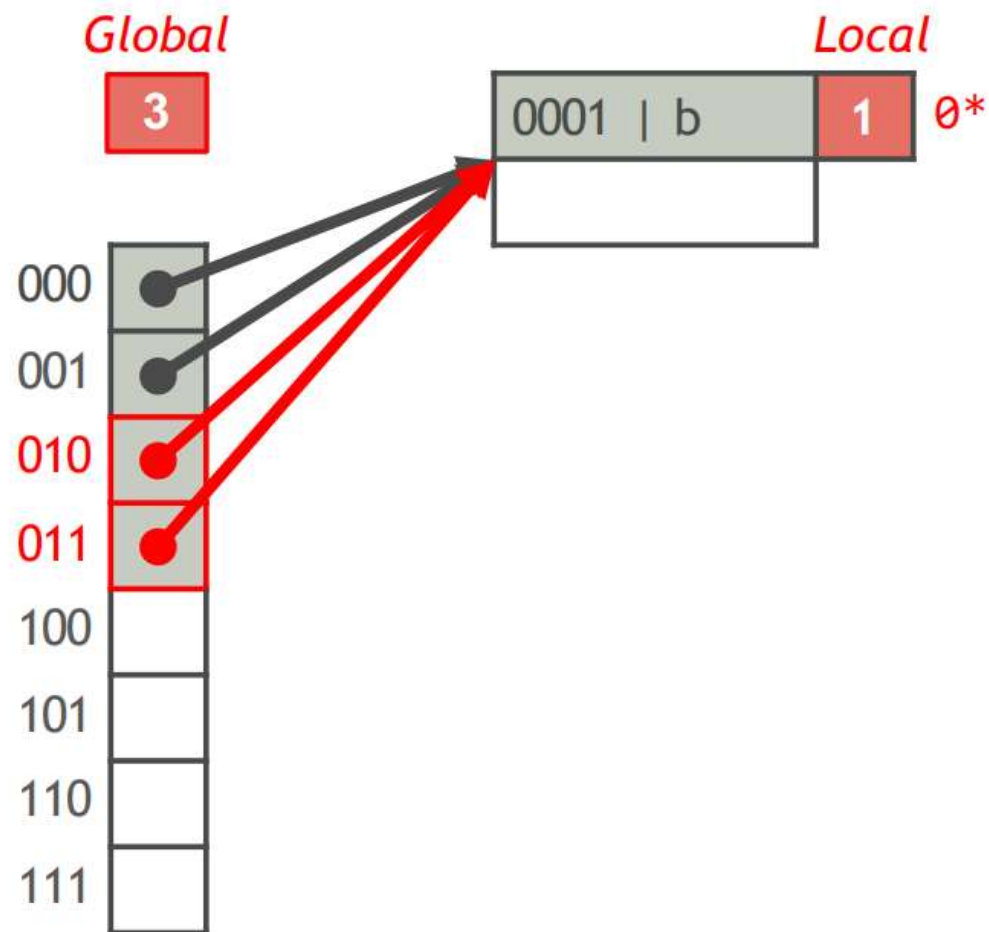
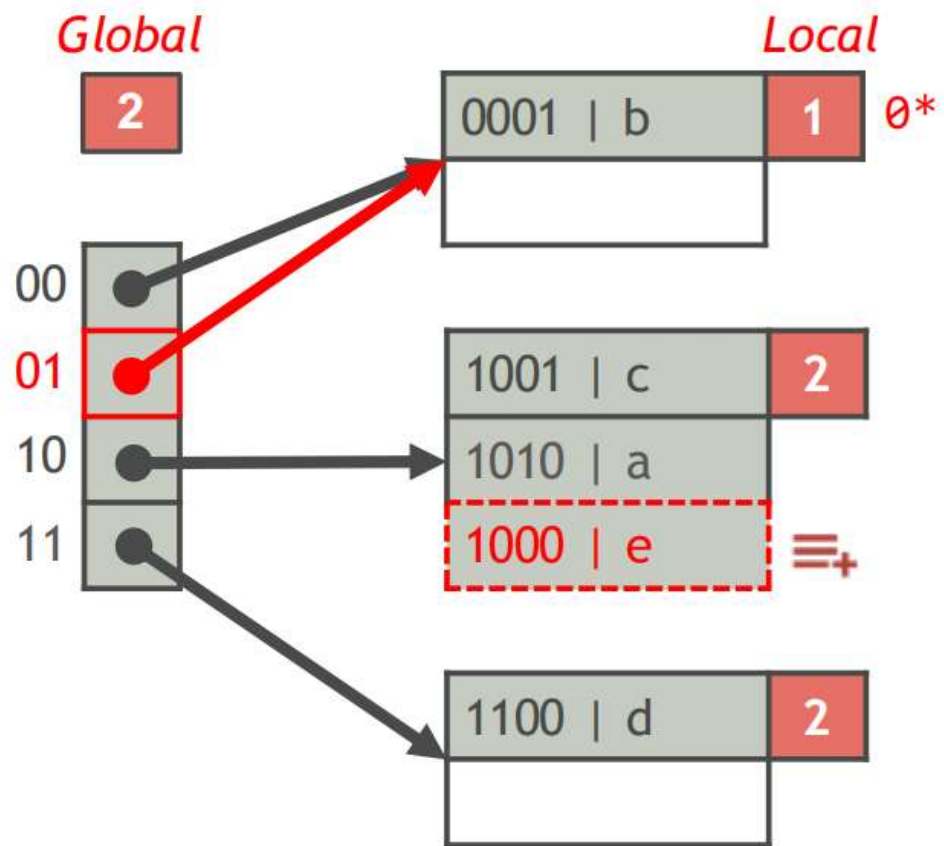
■ 可扩展哈希表插入：示例

示例：K = e, hash(e) = 1000



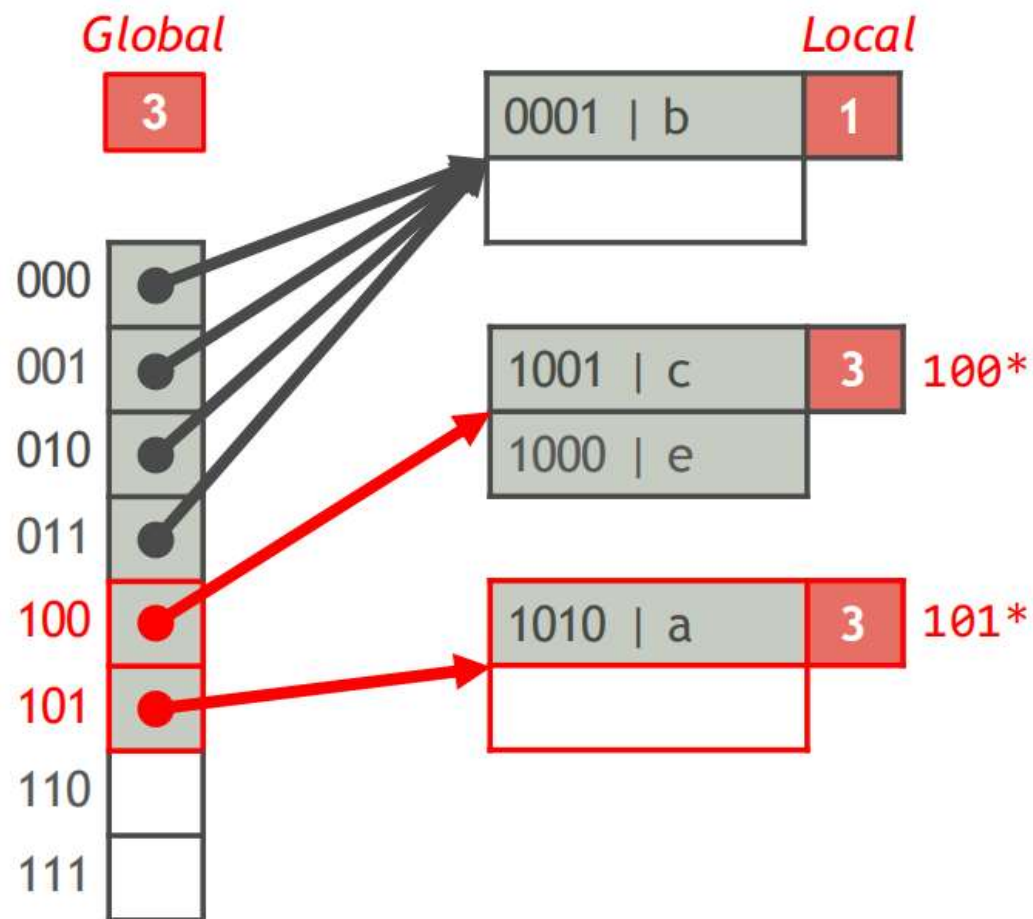
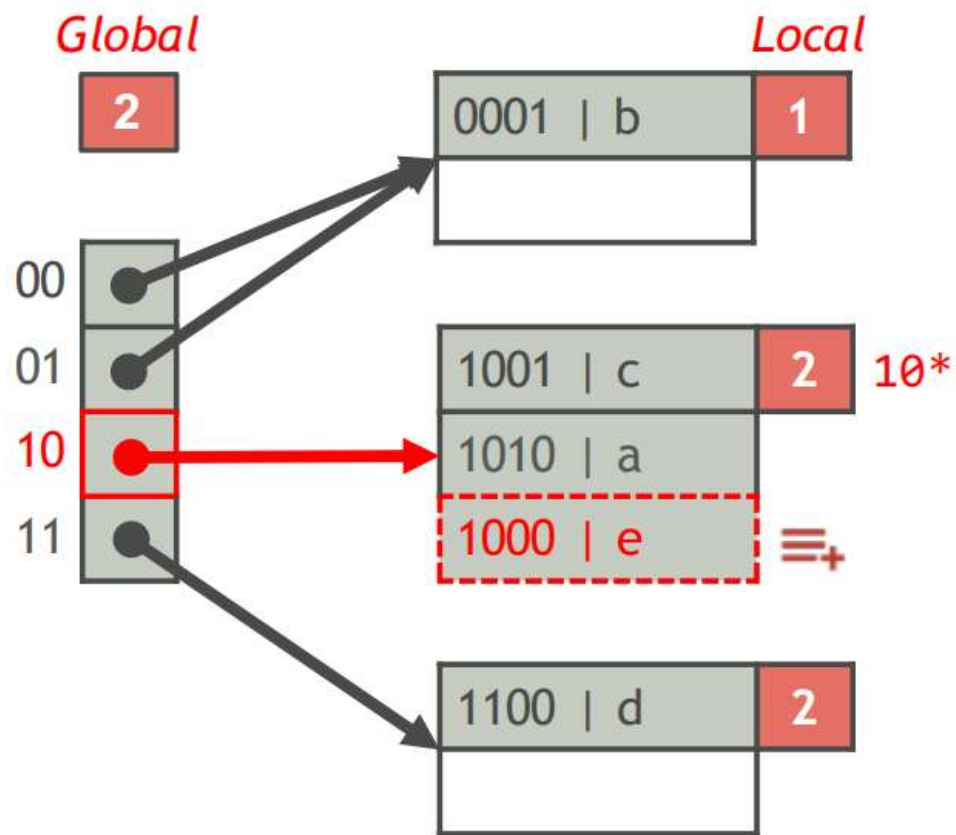
■ 可扩展哈希表插入：示例

示例：K = e, hash(e) = 1000



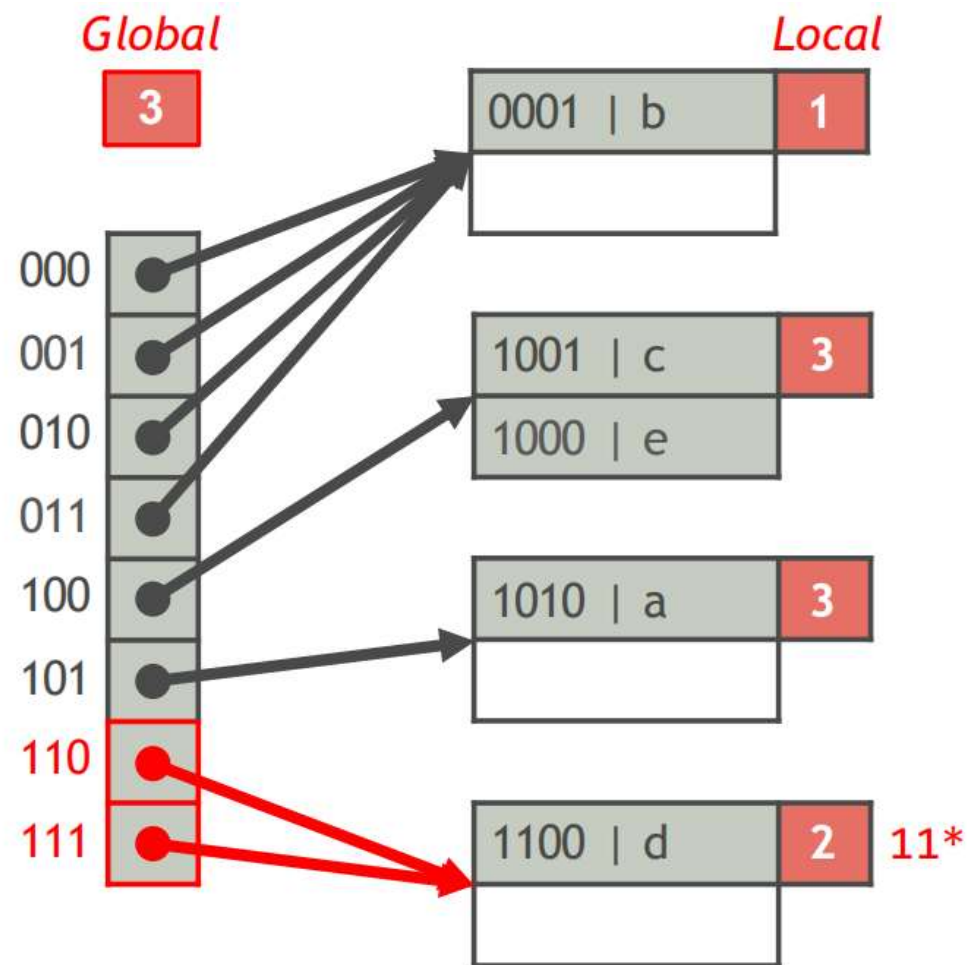
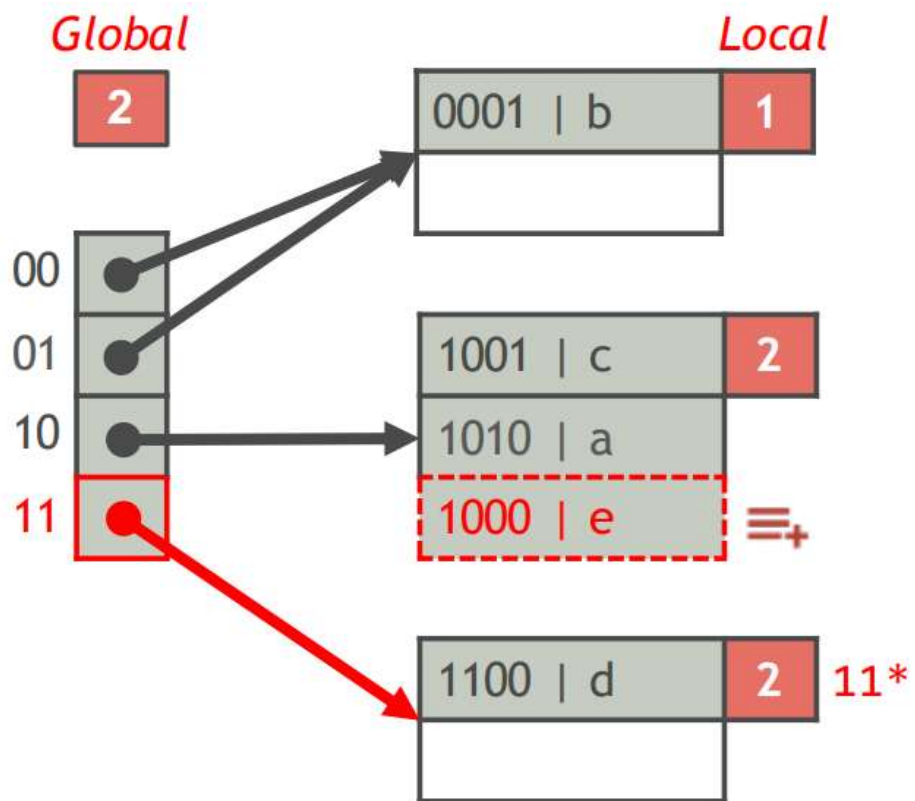
■ 可扩展哈希表插入：示例

示例：K = e, hash(e) = 1000



■ 可扩展哈希表插入：示例

示例：K = e, hash(e) = 1000

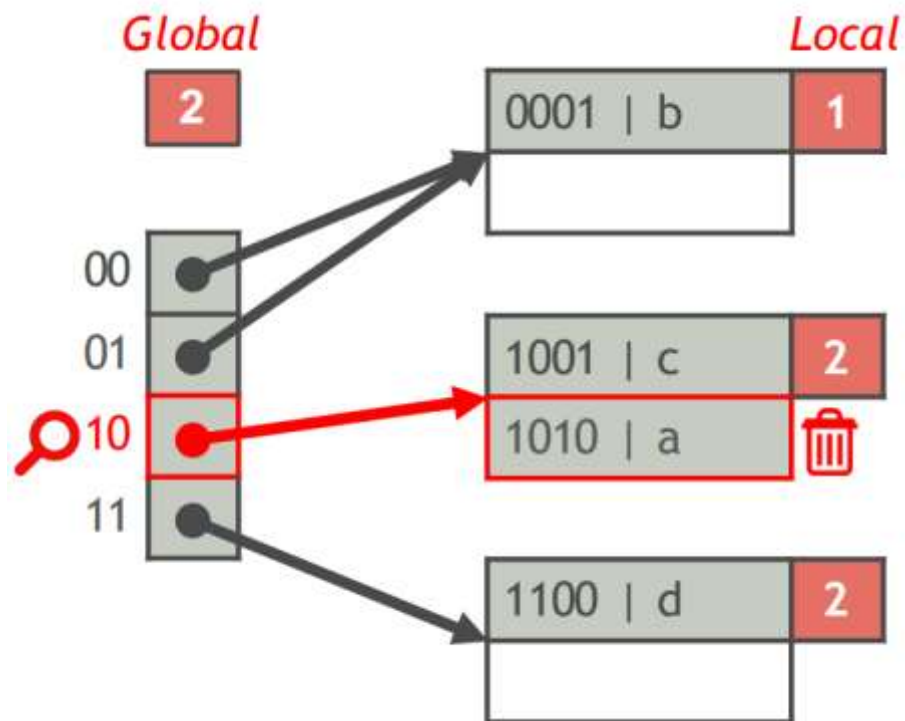


■ 可扩展哈希表删除

删除关键字为K的索引条目

- 找到条目所属的页
- 从页中删除条目

例如: $K = a$, $\text{hash}(a) = 1010$



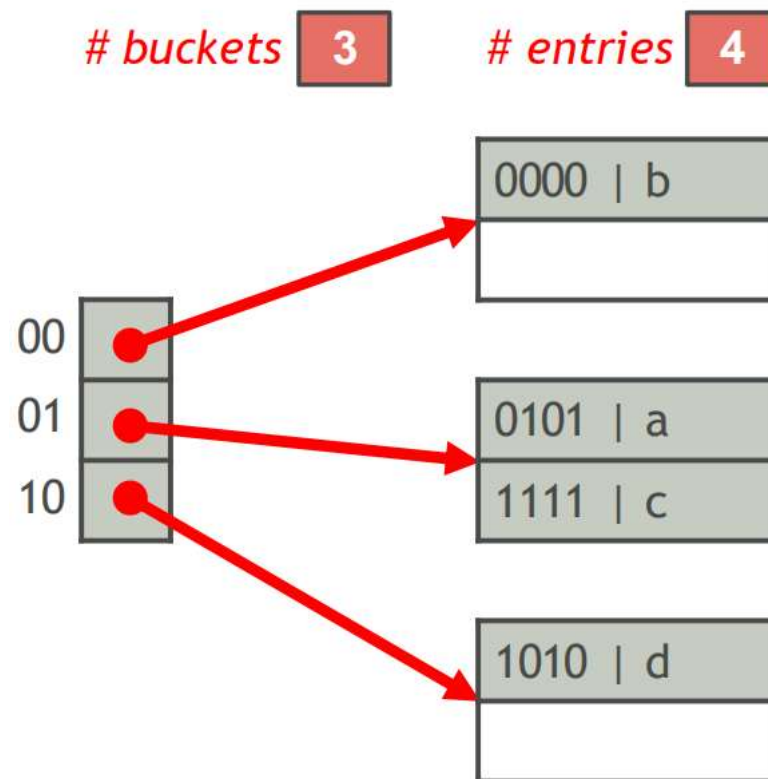
线性哈希表

■ 线性哈希表

一个线性哈希表由 n 个桶组成

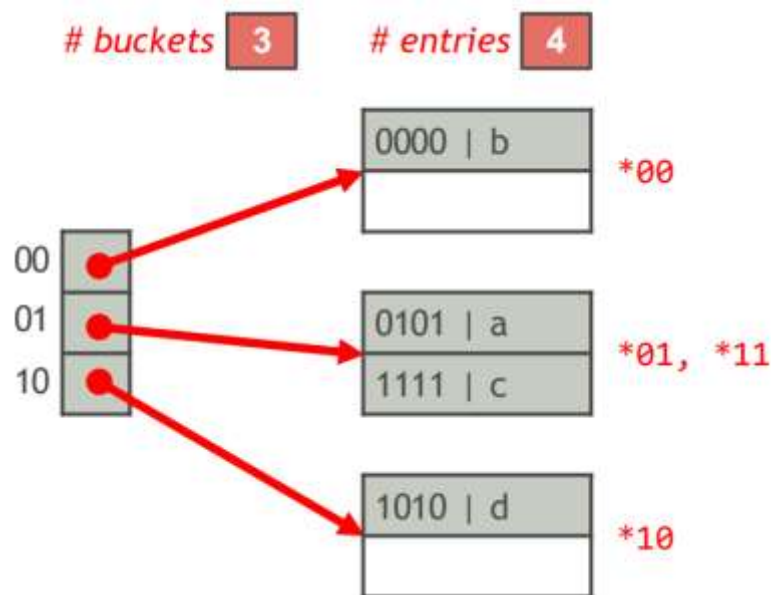
- 每个桶都保留指向包含桶内索引条目的页面的链接列表的指针
- 假设每个页面最多可以容纳 b 个索引条目。线性哈希表存储最多 θbn 个条目，其中 $0 < \theta < 1$ 是一个阈值

例如： $b = 2$, $\theta = 0.85$



■ 哈希方案 (续)

- 桶从 0 到 $n-1$ 编号
- 设 $m = 2^{\lfloor \log_2 n \rfloor}$, 其中 $m \leq n < 2m$
- 如果 $\text{hash}(K) \bmod 2m < n$, 则关键字为 K 的索引条目属于桶 $\text{hash}(K) \bmod 2m$; 否则, 它属于桶 $\text{hash}(K) \bmod m$
- 例如: $\text{hash}(a) = 0101$, $\text{hash}(b) = 0000$, $\text{hash}(c) = 1111$, $\text{hash}(d) = 1010$

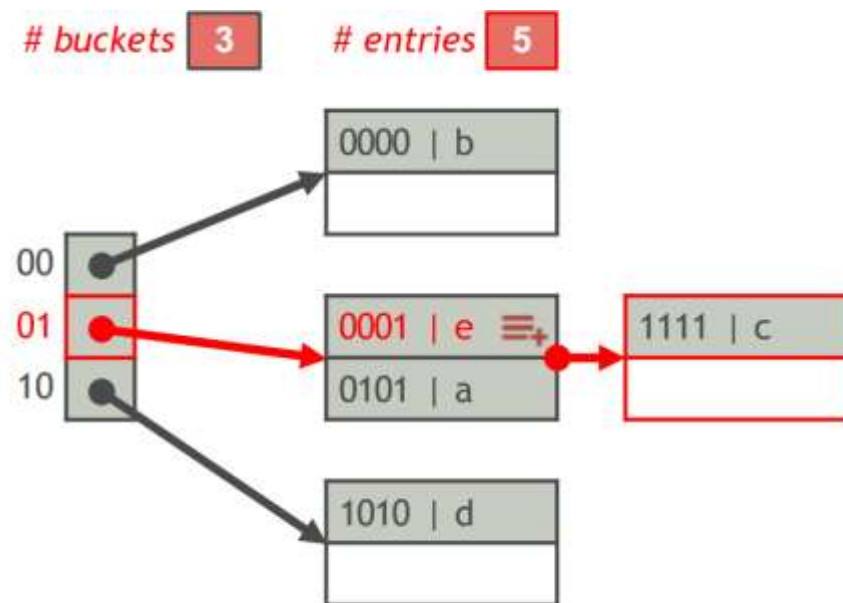
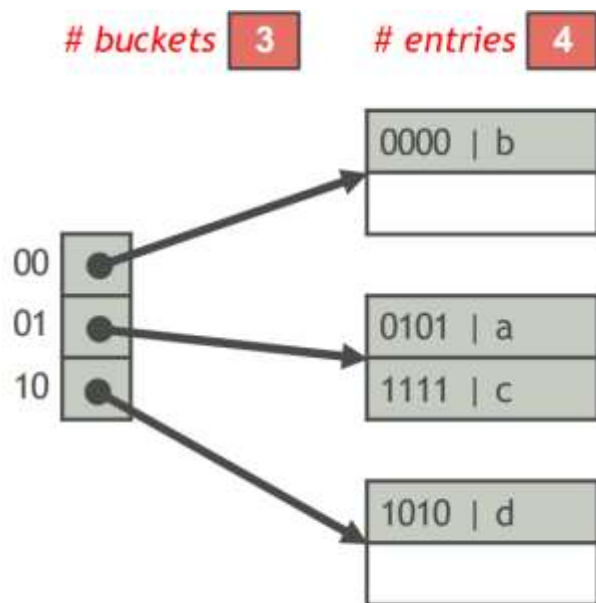


■ 线性哈希表插入

插入一个关键字为 K 的索引条目

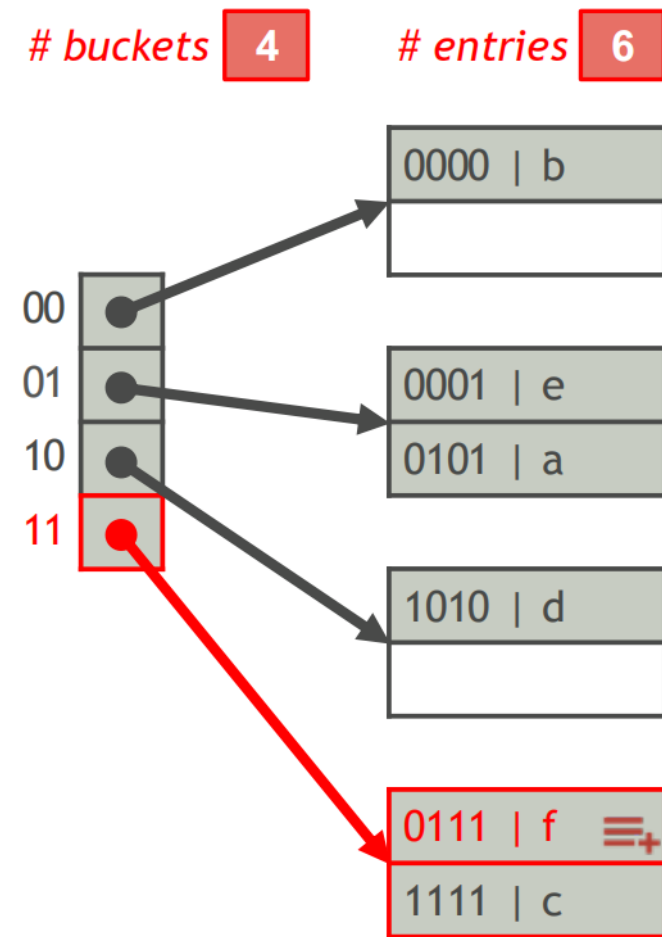
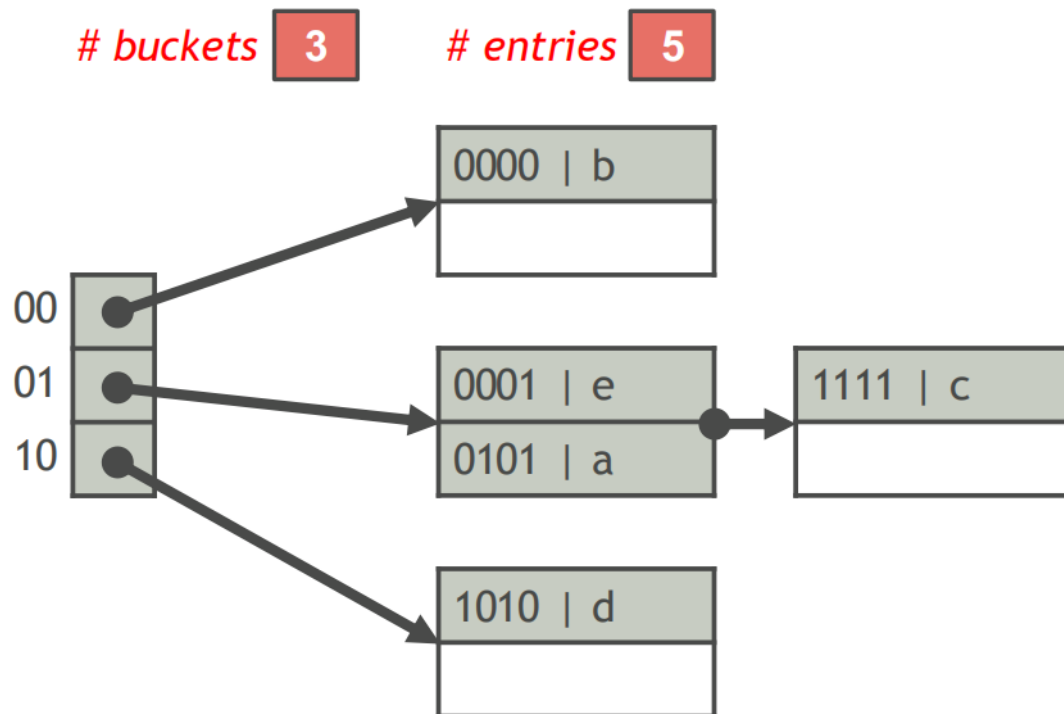
- 将条目插入属于它的桶 B 中
- 将 # 条目的数量增加1
- 如果 # 条目 $\leq \theta bn$, 则完成! 否则, 增加 # 桶数 1 并重新分配 B 中的条目

例如: $\text{hash}(e) = 0001$, $\theta = 0.85$



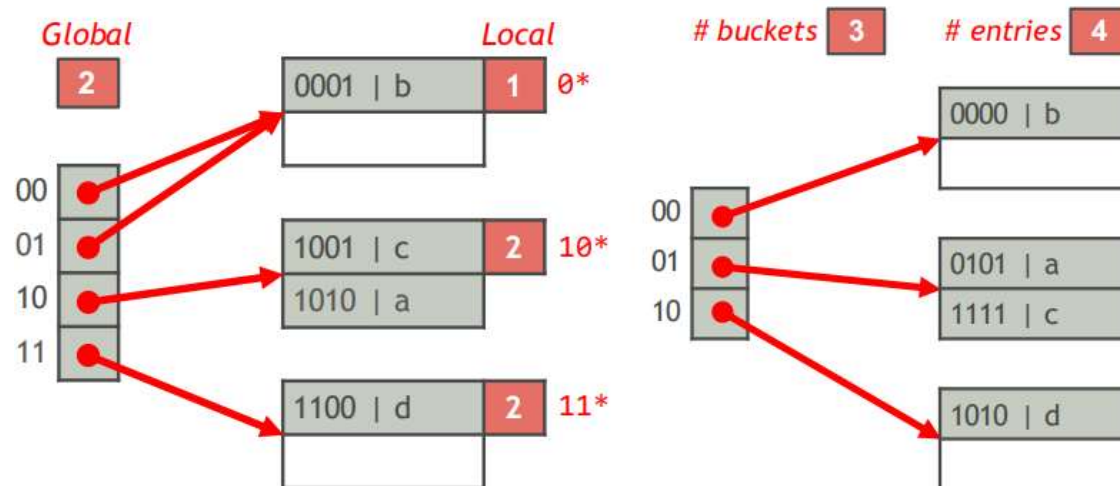
■ 线性哈希表插入 (续)

例如: $\text{hash}(f) = 0111$, $\theta = 0.85$



■ 可扩展哈希表 vs 线性哈希表

	Extensible hash tables	Linear hash tables
# Buckets	2^{global_depth}	n
Bucket pages	A bucket points to a single page	A bucket points to a linked list of pages
Hashing scheme	The first $global_depth$ bits of $hash(K)$	$hash(K) \bmod 2^m$ or $hash(K) \bmod m$
Page split condition	A page overflows	$\#entries > \theta bn$
Hash table expansion	# buckets is doubled ($global_depth$ is increased by 1)	# buckets is increased by 1





概述



B+ 树索引结构



哈希索引结构